

Diseño y Programación Orientada a Objetos
ISIS-1226
Board Game Café

Documento de Diseño e Implementación

Proyecto 2 - Entrega única

Santiago Ramírez Madrigal 202515057
Valentina Salgado Pineda 202511033
Isabella Casallas Abril 202510253
Daniel Felipe Nova Pulido 202212669

Universidad de los Andes
2026

1. Contexto y alcance

Nuestro objetivo es un sistema de software para la gestión de un Board Game Café: un establecimiento cuya propuesta combina la venta de cafetería con el préstamo y venta de juegos de mesa, una dinámica popularizada en Corea a comienzos del siglo XXI. El sistema se encarga de la administración de clientes, empleados e inventarios.

En este documento se describen las decisiones de diseño e implementación del sistema como parte de la segunda entrega, basados en el análisis del mundo presentados en la primera entrega. El análisis de dominio (el mundo), el modelo de dominio inicial (el diagrama presentado) y los requerimientos identificados fueron desarrollados en la Entrega 1 del mismo proyecto y sirven como punto de partida para las decisiones y reformas realizadas a partir de las recomendaciones aquí documentadas.

En primer lugar, hablaremos de nuestros objetivos en esta entrega (principalmente basándonos en el cumplimiento de los requerimientos funcionales) al igual que los no-objetivos para estabilizar los límites de nuestra implementación.

1.2 Objetivos del sistema

- Gestionar préstamo y devolución de juegos con validación completa (con validación completa nos referimos a requisitos como edad, disponibilidad, jugadores, etc.)
- Registrar ventas de juegos y alimentos con todos sus detalles (como los impuestos de cada uno).
- Administrar inventario físico separado para préstamo y para venta.
- Gestionar reservas de mesas con control de capacidad máxima del café.
- Administrar turnos semanales de empleados y solicitudes de cambio con aprobación del administrador.
- Soportar programa de fidelidad: acumulación de puntos y redención como descuento.
- Permitir que empleados sugieran platillos y que el administrador los apruebe o rechace.
- Soportar la creación y gestión de torneos de juegos (amistosos y competitivos), incluyendo inscripciones, cupos para fanáticos, premios y restricciones por rol de usuario.

1.3 No-objetivos

- No se implementa interfaz gráfica en esta entrega.
- No se procesan “pagos” electrónicos, es decir, la transferencia como tal.
- No se validan entradas en consola.
- No se maneja concurrencia ni acceso multiusuario simultáneo.
- No se implementa persistencia específica para los torneos en esta entrega (los torneos viven en memoria durante la sesión).

2. Referencia al análisis del mundo entrega 1

El análisis del dominio (incluyendo la identificación de entidades, atributos, relaciones y el modelo de dominio inicial) fue desarrollado y entregado en la Entrega 1. Dicho documento contiene la justificación de cada entidad identificada en el dominio del Board Game Café, sus relaciones de herencia y asociación, y una primera propuesta de diagrama de clases, sin embargo, se realizaron los debidos cambios mencionados por el profe para analizar de mejor forma los diagramas requeridos en esta entrega (como por ejemplo los cambios de las relaciones).

Por lo tanto, el modelo de dominio presentado en la Entrega 1 es el punto de partida del diseño de esta entrega. Dado que dicha entrega aún se encuentra en proceso de calificación y se realizaron cambios gracias a las recomendaciones del profesor, las decisiones tomadas en este documento pueden diferir del análisis inicial como resultado del proceso iterativo de diseño.

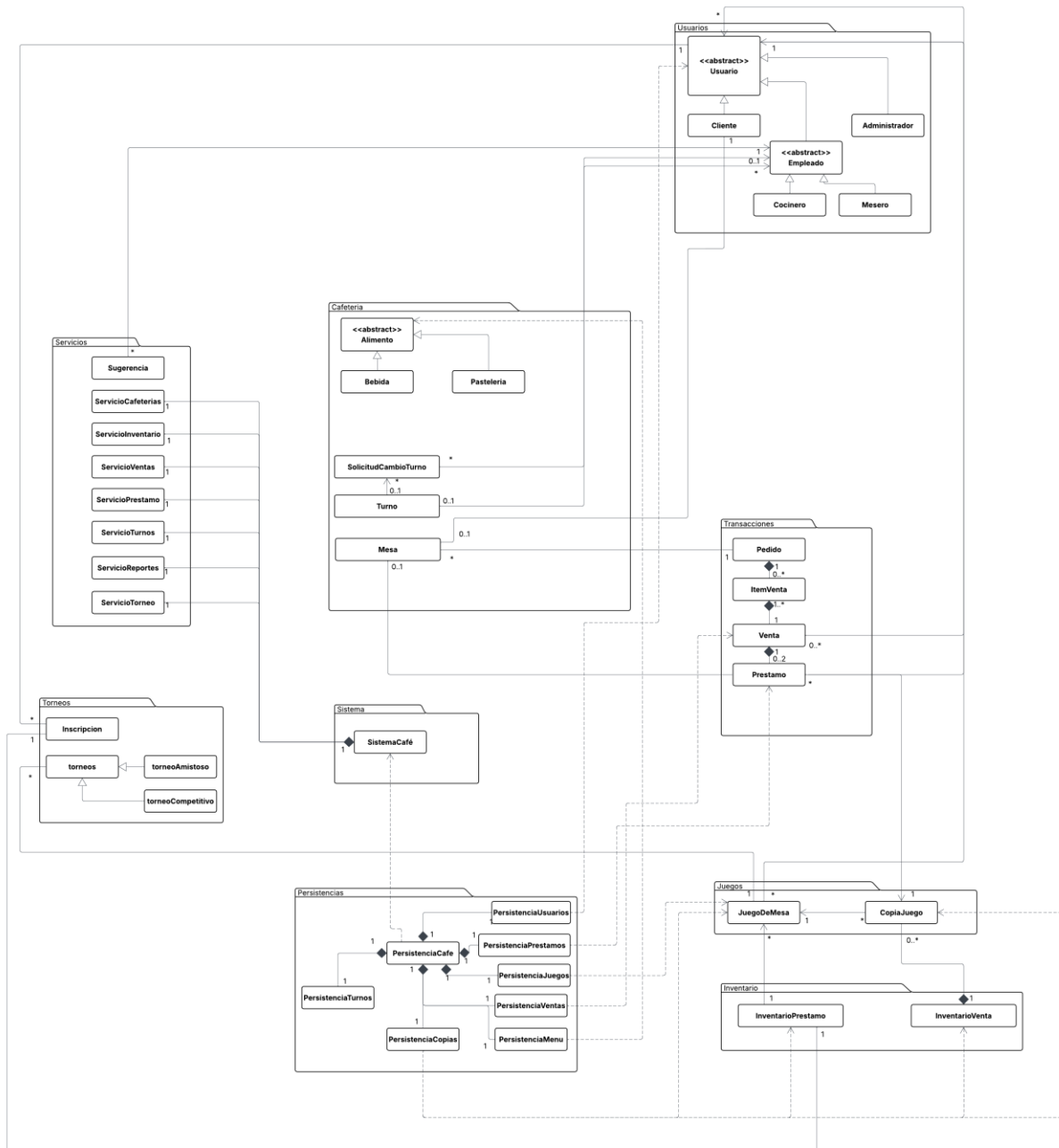
Sin embargo, los actores del sistema identificados en el análisis son los mismos que estructuran este documento de diseño, resumidos en la siguiente tabla:

Actor	Rol principal en el sistema
Cliente	Reserva mesas, pide juegos prestados, ordena alimentos, compra juegos, acumula y redime puntos, guarda favoritos.
Mesero	Atiende mesas, entrega juegos, enseña juegos difíciles, puede comprar y pedir préstamos fuera de turno, sugiere platillos.
Cocinero	Prepara alimentos, puede comprar y pedir préstamos fuera de turno, sugiere platillos. No cubre rol de mesero.
Administrador	Gestiona inventarios, turnos, menú, sugerencias, historial e informes. No puede pedir préstamos ni ordenar alimentos.

La anterior tabla, sirve como referencia de nuestra implementación en esta entrega.

3. Diagrama de clases — alto nivel

Esta vista muestra todas las clases del sistema con sus relaciones de herencia y asociación, sin atributos ni métodos. Su propósito es dar una perspectiva general de la arquitectura antes de entrar en el detalle, facilitando la comprensión de cómo se organizan los paquetes y como se relacionan entre sí.



usuarios	Usuario (abs.), Cliente, Empleado (abs.), Mesero, Cocinero, Administrador
juegos	JuegoDeMesa, CopiaJuego + enums: Categoria, EstadoCopia
cafeteria	Alimento (abs.), Bebida, Pasteleria, Mesa, Turno, SolicitudCambioTurno, Sugerencia
transacciones	Venta, ItemVenta, Pedido, Prestamo
inventario	InventarioPrestamo, InventarioVenta
Servicios	ServicioPrestamo, ServicioVentas, ServicioCafeteria, ServicioTurnos, ServicioInventario, ServicioReportes
persistence	PersistenciaCafe, PersistenciaUsuarios, PersistenciaJuegos, PersistenciaCopias, PersistenciaMenu, PersistenciaTurnos, PersistenciaPrestamos, PersistenciaVentas
sistema	SistemaCafe — coordina todos los servicios y es el punto de entrada único del sistema.
torneos	torneos (abs.), torneoAmistoso, torneoCompetitivo, Inscripcion

3.1 Justificación del diagrama de alto nivel

El diagrama de alto nivel permite apreciar de forma inmediata la estructura general del sistema sin perderse en los detalles de implementación. Las decisiones de diseño más relevantes visibles en esta vista son:

Jerarquía de usuarios centralizada en Usuario.

Toda la autenticación y los favoritos se manejan en la clase base. Los puntos de fidelidad están definidos en las subclases concretas (Cliente y Empleado) porque cada una los gestiona de forma independiente. Las subclases especializan únicamente lo que les corresponde: Cliente agrega la mesa y el historial de compras; Empleado agrega el turno y el código de descuento; Mesero agrega los juegos que conoce; Administrador no agrega estado, pero tiene permisos distintos según requerimientos funcionales.

Separación entre catalogo conceptual y copia física.

JuegoDeMesa representa el concepto del juego (nombre, reglas, categoría); CopiaJuego es el ejemplar físico con estado y disponibilidad. Esta separación permite rastrear cada copia individualmente sin duplicar datos del catálogo, y es la base del sistema de préstamos.

Dos inventarios independientes.

InventarioPrestamo y InventarioVenta son estructuras distintas. El primero gestiona copias físicas para préstamo; el segundo gestiona stock y precios para venta. El Administrador puede mover unidades entre ambos, operación coordinada por ServicioInventario.

Cafetería como subsistema separado.

Alimento, Bebida, Pastelería, Mesa, Turno y SolicitudCambioTurno forman un subsistema propio dentro del paquete Cafetería. Mesa es el pivote que conecta los préstamos de juegos con los pedidos de comida, ya que ambas validaciones dependen de la composición del grupo sentado.

Capa de servicios como intermediaria.

Ningún usuario interactúa directamente con los inventarios, mesas o préstamos. Toda la lógica pasa por los servicios, que aplican las reglas de negocio antes de modificar el estado del sistema.

Persistencia encapsulada en su propio paquete.

El paquete Persistencias es completamente invisible para la capa de lógica. Solo SistemaCafe conoce a PersistenciaCafe, y esta delega en las persistencias especializadas. Si el mecanismo de almacenamiento cambia (de archivos planos a base de datos), ningún servicio necesita modificarse.

SistemaCafe como raíz del sistema.

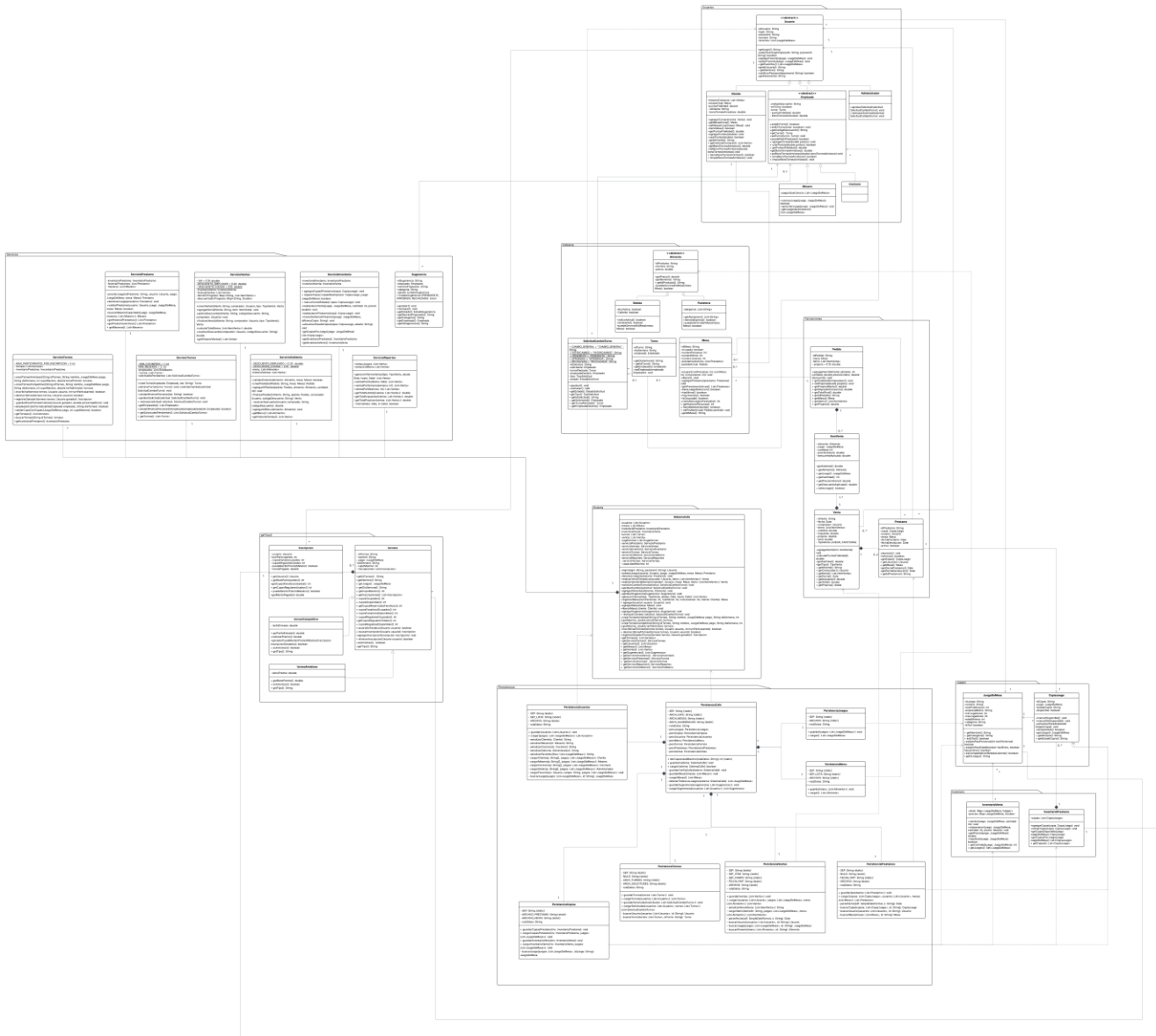
SistemaCafe es el único punto de entrada. Compone todos los servicios, gestiona la lista de usuarios y mesas, y es el único que conoce a PersistenciaCafe. Ninguna otra clase tiene acceso directo a todos los recursos del sistema, lo que centraliza el control y facilita la persistencia.

Nuevo paquete Torneos como subsistema independiente.

El paquete Torneos contiene torneos (abstracta), torneoAmistoso, torneoCompetitivo e Incripcion. La jerarquía usa polimorfismo para diferenciar el comportamiento de cada tipo: torneoAmistoso guarda un bonoPremio porcentual para el ganador, mientras que torneoCompetitivo calcula el premio en metálico como la suma de lo pagado por los clientes inscritos. Incripcion registra los cupos usados (fanáticos vs. regulares), el monto pagado y si el inscrito puede recibir premio metálico. ServicioTorneo coordina toda la lógica de inscripción y es el único componente que conoce este paquete, respetando la misma separación de responsabilidades del resto del sistema. Importante: las clases de prueba (JUnit) y de interfaz de consola no aparecen en el diagrama de clases principal porque no forman parte del dominio del sistema; son herramientas de validación y de interacción externas.

4. Diagrama de clases — diseño detallado

El siguiente diagrama incluye todas las clases con sus atributos, métodos, tipos de datos, relaciones y multiplicidades. Es la referencia principal para la implementación en Java.



4.1 Descripción y justificación de clases

Jerarquía de usuarios

Clase	Justificación
Usuario (abstract)	Base comun de todos los usuarios: idUsuario, login, password, nombre, puntosFidelidad y favoritos. Abstracta porque nunca existe un usuario genérico — siempre es un tipo concreto. Centraliza autenticacion y favoritos para no duplicar lógica.
Cliente	Agrega idCliente, puntosFidelidad propio, mesaActual e historialCompras. La referencia a Mesa permite validar prestamos y pedidos según la composición del grupo (niños, jóvenes). historialCompras es necesario para la contabilidad y el programa de fidelidad, mientras que bonoTorneoAmistoso permite guardar el premio de descuento obtenido al ganar un torneo amistoso.

Empleado (abstract)	Agrega codigoDescuento (20% propio / 10% compatible), enTurno, turno y puntosFidelidad. Abstracta porque Mesero y Cocinero tienen roles operativos distintos; además, permite validar si el empleado puede participar en torneos según su turno, y guardar el bono de descuento si gana un torneo amistoso.
Mesero	Agrega juegosQueConoce: List<JuegoDeMesa>. Es el único empleado que puede enseñar las reglas de juegos difíciles a los clientes. ServicioPrestamo consulta esta lista para determinar si hay un mesero capacitado disponible.
Cocinero	Sin atributos propios adicionales. Su existencia como clase concreta es esencial para que ServicioTurnos pueda verificar el mínimo de 1 cocinero por turno, ya que instanceof diferencia roles.
Administrador	Sin atributos propios. Hereda de Usuario. Tiene permisos especiales: gestionar inventario, aprobar turnos y ver informes. El enunciado establece que no puede pedir préstamos ni ordenar alimentos; esto se valida en SistemaCafe.

Juegos e inventario

Clase	Justificación
JuegoDeMesa	Catalogo conceptual del juego: idJuego, nombre, anioPublicacion, empresaMatriz, minJugadores, maxJugadores, edadMinima, categoria y dificil. Los métodos esAptoParaPersonas() y esAptoParaEdad() encapsulan las validaciones de préstamo aplicando el principio de informacion experta.
CopiaJuego	Unidad física de un juego: idCopia, estadoCopia (Nuevo / Bueno / Falta una pieza / Deteriorado / Desaparecido) y disponible. La separación de JuegoDeMesa permite rastrear cada ejemplar individualmente con su propio historial de préstamos y estado.
InventarioPrestamo	Gestiona la lista de CopiaJuego disponibles para préstamo. Provee getCopiaDisponible(juego). El administrador agrega, retira y mueve copias a través de ServicioInventario.
InventarioVenta	Gestiona stock y precios para venta mediante mapas JuegoDeMesa->cantidad y JuegoDeMesa->precio. El administrador reabastecer y el sistema descuenta unidades al concretar una venta.

Cafetería y operación

Clase	Justificación
Alimento (abstract)	Base del menu: idProducto, nombre y precio. El metodo puedeServirseEnMesa(mesa) es abstracto para obligar a cada subclase a implementar sus propias restricciones de servicio.
Bebida	Agrega alcoholica y caliente. Su implementacion de puedeServirseEnMesa() rechaza bebidas alcoholicas en mesas con menores, y calientes en mesas con juegos de Accion activos.
Pasteleria	Agrega alergenos: List<String>. puedeServirseEnMesa() siempre permite el servicio pero advierte con la lista de alergenos para que el cliente quede informado.

Mesa	Pivote de la operacion: idMesa, ocupada, numeroPersonas, numeroNinos, numeroJovenes, prestamosActivos (0..2) y pedidoActual. Centraliza la informacion del grupo para que todas las validaciones de prestamos y pedidos la consulten.
Turno	Asigna un empleado a un dia de la semana: idTurno, diaSemana y referencia al Empleado. Los turnos son fijos y semanales; ServicioTurnos los gestiona y valida los minimos de personal.
SolicitudCambioTurno	Registra peticiones de cambio: idSolicitud, tipo (CAMBIO_GENERAL / INTERCAMBIO), estado (PENDIENTE / APROBADA / RECHAZADA), solicitante, empleadoDestino (null si cambio general) y turnoAfectado.

Transacciones

Clase	Justificacion
Venta	Registro inmutable de una transaccion: idVenta, comprador, items, fecha, tipo (JUEGO / CAFETERIA), subtotal, impuesto y propina. El tipo determina el impuesto (IVA 19% tienda, consumo 8% cafeteria). Nunca se elimina del historial.
ItemVenta	Línea de detalle: cantidad, precioUnitario y descuentoAplicado. Tiene dos asociaciones mutuamente excluyentes {xor}: hacia Alimento (cafeteria) o hacia JuegoDeMesa (tienda). Composicion con Venta: no existe sin su venta.
Pedido	Estado transitorio antes de convertirse en Venta: agrupa items de cafeteria asociados a una Mesa y guarda el porcentaje de propina (10% por defecto, configurable). Se convierte en Venta al cerrarse.
Prestamo	Registro historico de cada prestamo: idPrestamo, copia, usuario, mesa (null si empleado), fechaPrestamo, fechaDevolucion y activo. El flag activo distingue prestamos vigentes de historicos. Nunca se elimina.

Servicios — capa lógica

Clase	Justificacion
ServicioPrestamo	Gestiona el ciclo completo de prestamo: validar elegibilidad del usuario, verificar disponibilidad de copia, buscar mesero capacitado para juegos dificiles, crear el Prestamo y registrarlo en el historial.
ServicioInventario	Coordina operaciones sobre ambos inventarios: agregar copias, mover de venta a prestamo, reparar copias (reemplaza una deteriorada por una del inventario de venta), marcar como robada y reabastecer.
ServicioCafeteria	Gestiona el menu y los pedidos: validar si un alimento puede servirse, agregar items, calcular impuesto al consumo (8%) y propina, y generar la Venta final. Registra historial de ventas de cafeteria. Ademas, permite aplicar el bono de descuento ganado en torneos amistosos como un descuento no acumulable con otros descuentos.
ServicioVentas	Gestiona ventas de la tienda: aplicar descuentos (20% empleado / 10% codigo compartido), calcular IVA (19%), descontar stock y registrar la Venta. Mantiene historial de ventas de tienda. Ademas, permite aplicar el bono de descuento ganado en torneos amistosos como un descuento no acumulable con el descuento de empleado o código compartido.

ServicioTurnos	Gestiona turnos semanales: crear y eliminar turnos, registrar solicitudes, validar que el cambio no deje el café por debajo del mínimo (1 cocinero, 2 meseros), aprobar o rechazar solicitudes.
ServicioReportes	Genera informes para el Administrador: filtrar por tipo (JUEGO / CAFETERIA), por fecha (diaria, semanal, mensual) y calcular totales de subtotal, impuestos y propinas de forma segregada.
Sugerencia	Propuesta de empleado para agregar un platillo: idSugerencia, nombrePropuesto, categoria y estado (PENDIENTE / APROBADA / RECHAZADA). El administrador la aprueba o rechaza.
ServicioTorneo	Gestiona los torneos del café: crea torneos amistosos y competitivos, valida cupos según las copias disponibles, controla inscripciones y desinscripciones, separa cupos fanáticos y regulares, verifica que empleados no estén en turno y registra ganadores con premio metálico o bono de descuento según el tipo de torneo.

Persistencia

Clase	Justificación
PersistenciaCafe	Coordinador principal. Orquesta el guardado y carga del sistema respetando el orden de dependencias entre entidades. Compone a todas las demás persistencias especializadas. Único componente de persistencia que SistemaCafe conoce.
PersistenciaJuegos	Serializa JuegoDeMesa en juegos.txt (idJuego nombre anio empresa minJ maxJ edadMin categoria dificil). Se carga primero porque CopiaJuego depende de ella.
PersistenciaCopias	Serializa CopiaJuego (copias_prestamo.txt) e InventarioVenta (inventario_venta.txt). Necesita juegos cargados para reconstruir referencias.
PersistenciaUsuarios	Serializa todos los tipos de usuario con discriminador (CLIENTE MESERO COCINERO ADMIN). Incluye favoritos y juegosQueConoce del mesero. Necesita juegos cargados.
PersistenciaMenu	Serializa Bebida y Pasteleria en menu.txt con discriminador (BEBIDA PASTELERIA). Reconstruye el menú completo al cargar.
PersistenciaTurnos	Serializa Turno en turnos.txt y SolicitudCambioTurno en solicitudes.txt. Necesita usuarios cargados para reconstruir referencias a empleados.
PersistenciaPrestamos	Serializa el historial completo de Préstamo en prestamos.txt. Necesita copias, usuarios y mesas cargados. Si el préstamo sigue activo, marca la copia como no disponible al cargar.
PersistenciaVentas	Serializa el historial de Venta en ventas.txt. Cada ítem con discriminador (J para juego, A para alimento). Necesita usuarios, juegos y menú cargados para reconstruir ítems.

Torneos

Clase	Justificación
torneos (abstract)	Clase base del subsistema de torneos. Centraliza idTorneo, nombre, juego, diaSemana, cupoMaximo e inscripciones. Define el cálculo de cupos

	reservados para fanáticos (20% redondeado hacia arriba) y los métodos de gestión de Incripcion. Abstracta porque nunca existe un torneo genérico: siempre es amistoso o competitivo.
torneoAmistoso	Agrega bonoPremio: double, el porcentaje de descuento que se otorga al ganador. No cobra tarifa de entrada. El descuento se guarda en el usuario ganador a través de ServicioTorneo.guardarBonoPremioAmistoso().
torneoCompetitivo	Agrega tarifaEntrada: double por participante. calcularPremio() suma el montoPagado de todas las inscripciones de clientes (los empleados no pagan y no suman al premio). ganadorPuedeRecibirPremioMetalico() verifica el flag de la Incripcion del ganador.
Incripcion	Registra la participación de un usuario en un torneo: numParticipantes (1–3), cuposFanaticosUsados, cuposRegularesUsados, puedeRecibirPremioMetalico y montoPagado. Su existencia permite rastrear cómo se distribuyeron los cupos y si el inscrito puede recibir el premio en metálico.

4.2 Relaciones del diagrama

Herencia:

Origen — clase hijo	Destino — clase padre
Empleado	Usuario
Cliente	Usuario
Administrador	Usuario
Mesero	Empleado
Cocinero	Empleado
Bebida	Alimento
Pasteleria	Alimento
torneoAmistoso	torneos
torneoCompetitivo	torneos

Composición:

Contenedor (rombo aquí — lado 1)	Contenido (punta aquí)	Multiplicidad en el lado del contenido
Mesa	Préstamo	0..2 (max 2 prestamos activos simultaneos)
Venta	ItemVenta	1..* (al menos 1 item por venta)
Pedido	ItemVenta	0..* (se acumulan antes de cerrar)
InventarioPrestamo	CopiaJuego	0..* (cero o mas copias fisicas)
PersistenciaCafe	PersistenciaJuegos	1
PersistenciaCafe	PersistenciaCopias	1

PersistenciaCafe	PersistenciaUsuarios	1
PersistenciaCafe	PersistenciaMenu	1
PersistenciaCafe	PersistenciaTurnos	1
PersistenciaCafe	PersistenciaPrestamos	1
PersistenciaCafe	PersistenciaVentas	1
SistemaCafe	ServicioPrestamo	1
SistemaCafe	ServicioInventario	1
SistemaCafe	ServicioCafeteria	1
SistemaCafe	ServicioVentas	1
SistemaCafe	ServicioTurnos	1
SistemaCafe	ServicioReportes	1
torneos	Inscripcion	0. (cero o mas inscripciones por torneo)
SistemaCafe	ServicioTorneo	1

Asociación:

Origen	Mult. origen	Destino	Mult. destino	Etiqueta / nota
CopiaJuego	*	JuegoDeMesa	1	atributo juego
Prestamo	*	CopiaJuego	1	copia prestada
Prestamo	*	Usuario	1	quien solicito el préstamo
Prestamo	*	Mesa	0..1	null si prestatario es empleado
Turno	*	Empleado	1	empleado asignado
Empleado	1	Turno	0..1	turno asignado; null si no tiene
SolicitudCambioTurno	*	Empleado	1	solicitante
SolicitudCambioTurno	*	Empleado	0..1	empleadoDestino; null si cambio general
SolicitudCambioTurno	*	Turno	0..1	turnoAfectado
Cliente	*	Mesa	0..1	mesaActual; null si no sentado
Cliente	1	Venta	0..*	historialCompras
Venta	*	Usuario	1	comprador
Mesero	*	JuegoDeMesa	*	juegosQueConoce — muchos a muchos
Usuario	*	JuegoDeMesa	*	favoritos — muchos a muchos
Sugerencia	*	Empleado	1	autor de la sugerencia

InventarioVenta	1	JuegoDeMesa	*	clave del mapa de stock y precios
torneos	*	JuegoDeMesa	1	Juego del torneo
Inscripcion	*	Usuario	1	Usuario Inscrito
ServicioTorneo	1	InventarioPrestamo	1	Valida copias disponibles para el cupo

Dependencia de uso:

Clase que usa	Clase usada	Razon
ServicioPrestamo	InventarioPrestamo	busca copias disponibles para prestar
ServicioPrestamo	Mesero	busca mesero capacitado para juegos dificiles
ServicioPrestamo	Prestamo	crea y retorna objetos Prestamo
ServicioInventario	InventarioPrestamo	recibido en constructor; opera sobre el
ServicioInventario	InventarioVenta	recibido en constructor; opera sobre el
ServicioInventario	CopiaJuego	repara, marca robada y actualiza estado
ServicioCafeteria	Alimento	gestiona el menu y valida servicio en mesa
ServicioCafeteria	Mesa	valida si un alimento puede servirse
ServicioCafeteria	Pedido	crea y cierra pedidos de cafetería
ServicioCafeteria	Venta	retorna la Venta al finalizar el pedido
ServicioVentas	InventarioVenta	recibido en constructor; descuenta stock
ServicioVentas	Venta	crea y retorna ventas de tienda
ServicioTurnos	Empleado	gestiona sus turnos y valida minimos de personal
ServicioTurnos	Turno	crea, modifica y elimina turnos
ServicioTurnos	SolicitudCambioTurno	aprueba o rechaza solicitudes de cambio
ServicioReportes	Venta	recibe listas de ventas y genera informes
Bebida	Mesa	puedeServirseEnMesa(): verifica menores y Accion
Pasteleria	Mesa	puedeServirseEnMesa(): advierte alergen
SistemaCafe	PersistenciaCafe	único cliente de la capa de persistencia
PersistenciaCafe	SistemaCafe	recibe el sistema en guardar() y cargar()

PersistenciaCafe	Mesa	guarda y carga mesas directamente
PersistenciaCafe	Sugerencia	guarda y carga sugerencias directamente
PersistenciaCafe	Empleado	reconstruye el autor de cada sugerencia
PersistenciaJuegos	JuegoDeMesa	serializa y deserializa el catálogo de juegos
PersistenciaCopias	CopiaJuego	serializa copias físicas del inventario
PersistenciaCopias	InventarioPrestamo	carga copias en el inventario al iniciar
PersistenciaCopias	InventarioVenta	carga stock y precios al iniciar
PersistenciaUsuarios	Usuario (y subclases)	serializa todos los tipos con discriminador de tipo
PersistenciaUsuarios	JuegoDeMesa	reconstruye favoritos y juegosQueConoce del mesero
PersistenciaMenu	Alimento / Bebida / Past.	serializa el menú con discriminador BEBIDA PASTELERIA
PersistenciaTurnos	Turno	serializa los turnos semanales
PersistenciaTurnos	SolicitudCambioTurno	serializa solicitudes pendientes de cambio
PersistenciaTurnos	Empleado	reconstruye la referencia empleado-turno al cargar
PersistenciaPrestamos	Prestamo	serializa el historial completo de prestamos
PersistenciaPrestamos	CopiaJuego	reconstruye que copia fue prestada
PersistenciaPrestamos	Usuario	reconstruye quien realizo el préstamo
PersistenciaPrestamos	Mesa	reconstruye en que mesa se realizó el préstamo
PersistenciaVentas	Venta	serializa el historial de ventas completo
PersistenciaVentas	ItemVenta	serializa cada linea de detalle de la venta
PersistenciaVentas	Usuario	reconstruye el comprador de cada venta
PersistenciaVentas	JuegoDeMesa	reconstruye items de tipo juego en la venta
PersistenciaVentas	Alimento	reconstruye items de cafetería en la venta
ServicioTorneo	torneos / torneoAmistoso / torneoCompetitivo	crea y opera sobre todos los tipos de torneo
ServicioTorneo	Inscripcion	crea inscripciones y las registra en el torneo
ServicioTorneo	InventarioPrestamo	valida que haya copias suficientes para el cupo

ServicioTorneo	Empleado	verifica turno del empleado para inscripción
SistemaCafe	ServicioTorneo	único cliente del servicio de torneos
torneoCompetitivo	Inscripcion	calcula el premio sumando montoPagado de clientes

Nota:

Las clases de prueba JUnit (ServicioVentasTest, SistemaCafeTest, ClienteTest, etc.) y las clases de interfaz de consola no aparecen en el diagrama de clases de dominio. Los diagramas UML de diseño modelan el dominio del sistema, no su infraestructura de pruebas ni su capa de presentación. Fue confirmado previamente.

5. Requerimientos funcionales

Los requerimientos funcionales identificados en la Entrega 1 fueron 32 en total. Para esta entrega, se consolidaron en 23 requerimientos agrupando aquellos que representan el mismo “flujo” para distintos actores o “actores” del mismo proceso, esto debido a las recomendaciones dadas por el profesor, así mismo, para ayudar a las pruebas de la implementación. A continuación, se documenta qué se fusionó y por qué.

5.1 Tabla de “consolidación” (de 32 a 27 RF)

En este caso, se fusionaron cuando se tenía el “mismo” proceso pero con diferentes permisos o actores.

RFs originales	RF consolidado	Justificación
RF originales fusionados	RF consolidado	Justificación de la fusión
RF15 (cliente pide prestado) con RF25 (empleado pide prestado)	RF06 - Pedir juego en préstamo	El flujo de validación es el mismo (disponibilidad, jugadores, edad, categoría). La diferencia es el actor: cliente necesita mesa, empleado necesita estar fuera de turno.
RF14 (cliente devuelve juegos) con RF26 (empleado devuelve juego)	RF07 - Devolver juego prestado	El proceso de devolución es idéntico: marcar préstamo inactivo, registrar fecha, liberar copia. El actor no cambia el flujo.
RF17 (cliente compra juego) con RF27 (empleado compra juego)	RF10 - Comprar juego del inventario de venta	El proceso de venta es el mismo. La diferencia es el descuento aplicado (0% cliente, 20% empleado, 10% código compartido).
RF18 (cliente pide alimento) con RF28 (empleado pide alimento)	RF11 - Pedir alimento del menú	Las validaciones son idénticas (alcohólico, caliente, alérgenos). La diferencia es el descuento del empleado (20%).
RF20 (cliente favorito) con RF29 (empleado favorito)	RF12 - Guardar juego como favorito	El comportamiento es exactamente idéntico para ambos actores. No existe ninguna diferencia en el flujo ni en las validaciones.
RF22 (solicitar cambio) con RF23 (solicitar intercambio)	RF14 - Solicitar cambio de turno	Ambos son solicitudes de modificación de turno que siguen el mismo flujo: verificar mínimos → crear RegistroTurnos con tipo → quedar PENDIENTE. La diferencia es el campo tipo (CAMBIO vs INTERCAMBIO).
RF7 (aprobar cambio) con RF8 (construir turnos admin)	RF02 - Gestionar turnos del personal	Ambos son acciones del administrador sobre los turnos. Uno aprueba solicitudes, el otro modifica

		directamente. Son parte del mismo proceso de administración de turnos.
RF9 (agregar alimento) con RF10 (aprobar sugerencia)	RF04 - Gestionar menú del café	Ambos resultan en la misma acción: agregar un platillo al menú. La diferencia es el origen (creación directa vs. aprobación de sugerencia). Son parte del mismo proceso de gestión de menú.
RF30 (mesero atiende mesas) con RF31 (mesero entrega juego)	RF22 - Responsabilidades operativas del mesero	Ambos describen el rol operativo del mesero durante su turno. Se pueden describir como un único requerimiento de 'atención y entrega'.

5.2 Requerimientos funcionales consolidados

Administrador:

RF	Nombre	Descripción
RF01	Gestionar inventario de juegos	Consultar, agregar unidades, mover juegos entre inventario de préstamo y venta, reabastecer stock. Incluye ver el estado de cada copia y la disponibilidad actual.
RF02	Gestionar turnos del personal	Construir y modificar directamente los turnos semanales, y aprobar o rechazar solicitudes de cambio de turno enviadas por los empleados. Garantiza siempre mínimo 1 cocinero y 2 meseros por día.
RF03	Gestionar estado de copias de juegos	Cambiar el estado físico de copias (FALTA_PIEZA, DETERIORADO), marcar copias como DESAPARECIDO cuando no se devuelven, y reparar copias dañadas reemplazándolas con unidades del inventario de venta.
RF04	Gestionar menú del café	Agregar nuevos alimentos directamente o aprobar/rechazar sugerencias de empleados. Ambas acciones resultan en la disponibilidad de un nuevo platillo en el menú.
RF05	Consultar historial e informes	Consultar el historial completo de préstamos (activos e históricos) y generar informes de ventas filtrados por tipo (TIENDA/CAFETERIA) y fecha (día/semana/mes), con desglose de subtotal, impuestos y propinas.
RF24	Crear torneo	El administrador crea torneos amistosos (con bono de descuento) o competitivos (con tarifa de entrada y premio en metálico). Define juego, día de la semana y cupo máximo. El sistema valida que las copias del inventario de préstamo soporten el cupo definido.
RF25	Registrar ganador del torneo	El administrador registra al ganador de un torneo. Si es amistoso, el bono de descuento se guarda en el usuario. Si es competitivo y el ganador puede recibirlo, se informa el premio en metálico calculado.

Clientes y empleados (Es decir, los requerimientos que estos dos comparten):

RF	Nombre	Descripción
RF06	Pedir juego en préstamo	Cliente (con mesa, máx. 2 activos) o empleado (fuera de turno, sin clientes en el café) puede solicitar una copia disponible. El sistema valida disponibilidad, jugadores, edad, categoría vs. bebidas y si el juego es difícil.
RF07	Devolver juego prestado	Cualquier usuario con préstamo activo puede devolverlo. El sistema marca el préstamo como inactivo, registra la fecha y libera la copia para otros usuarios.
RF08	Consultar catálogo de juegos	Cualquier usuario puede ver el catálogo completo con el indicador de disponibilidad en tiempo real de cada juego (disponible / no disponible para préstamo en este momento).
RF09	Guardar y consultar juegos favoritos	Clientes y empleados pueden marcar juegos como favoritos. Esta información se usará en campañas y promociones futuras.
RF10	Comprar juego del inventario de venta	Clientes y empleados pueden comprar juegos. Se aplica IVA 19%. Empleados tienen descuento automático del 20%; clientes pueden usar código compartido de empleado para 10% (no acumulables). Genera puntos de fidelidad (1% del total).
RF11	Pedir alimento del menú	Clientes y empleados pueden pedir bebidas o pastelería. El sistema valida: alcohólico con menores, bebida caliente con juego Acción, informa alérgenos en pastelería. Aplica impuesto al consumo 8% + propina sugerida 10% (editable). Empleados con 20% de descuento.
RF12	Usar puntos de fidelidad	Clientes y empleados pueden aplicar puntos acumulados como descuento en cualquier compra. 1 punto = 1 peso de descuento. El sistema actualiza el saldo de puntos automáticamente.
RF26	Inscribirse a un torneo	Clientes y empleados pueden inscribirse a un torneo indicando el número de participantes (1–3). El sistema valida: cupos disponibles, que el usuario no esté ya inscrito, que los empleados no tengan turno ese día, y distribuye cupos entre la reserva de fanáticos (20% redondeado arriba) y cupos regulares. En torneos competitivos, los empleados participan gratis, pero sin premio en metálico.
RF27	Desinscribirse de un torneo	Cualquier usuario inscrito puede desinscribirse. Se eliminan todos los cupos tomados por esa inscripción.

Clientes (exclusivos):

RF	Nombre	Descripción
RF13	Reservar mesa	El cliente indica número de personas, niños (mayor de 5 años) y jóvenes (mayor de 18 años). El sistema verifica que no se supere la capacidad máxima del café. Si hay cupo, asigna mesa y registra la reserva.
RF14	Liberar mesa	Al finalizar la visita, el cliente libera la mesa. El sistema marca todos los préstamos activos de esa mesa como devueltos y deja las copias disponibles.

Empleados (exclusivos):

RF	Nombre	Descripción
RF15	Consultar turnos propios	Cada empleado puede ver sus turnos asignados. No puede modificarlos directamente.
RF16	Solicitar cambio o intercambio de turno	El empleado puede solicitar eliminar un turno propio (CAMBIO) o intercambiarlo con otro empleado del mismo rol (INTERCAMBIO).
RF17	Sugerir nuevo platillo al menú	Cualquier empleado puede proponer un platillo indicando nombre y categoría (bebida/pastelería).

Mesero (específicos debido al rol):

RF	Nombre	Descripción
RF18	Enseñar juegos difíciles a clientes	Cuando se solicita un juego difícil, el sistema verifica si hay mesero en turno con ese juego en su lista de juegosAprendidos. Si hay, se registra su participación. Si no hay, el préstamo se permite bajo advertencia.
RF19	Aprender juegos difíciles	El mesero puede agregar juegos a su lista juegosAprendidos para quedar habilitado para enseñarlos a los clientes.

Cocinero (específico debido al rol):

RF	Nombre	Descripción
RF20	Preparar alimentos del menú	El cocinero queda asociado a los alimentos que prepara. Un alimento puede ser preparado por distintos cocineros según el turno.

Sistema:

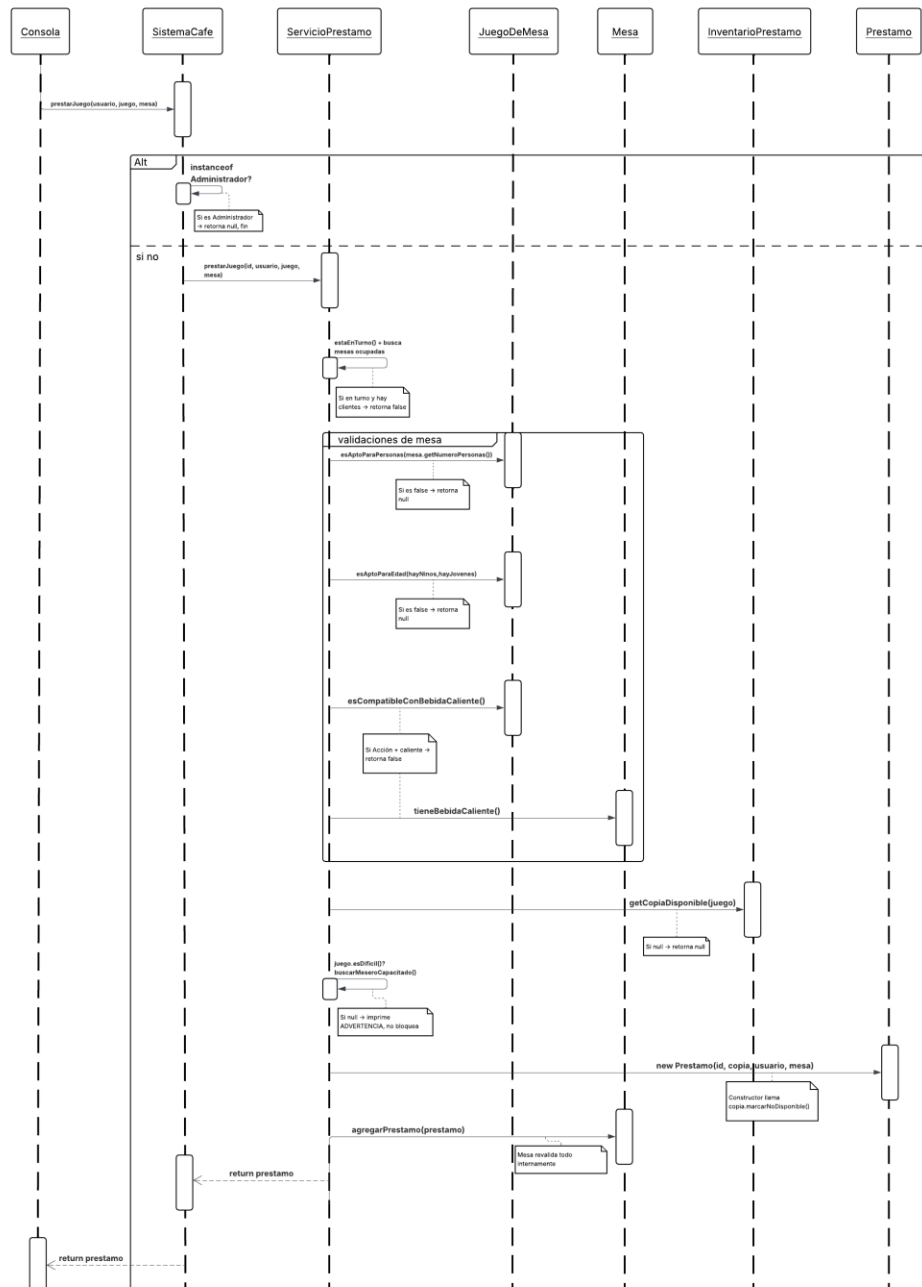
RF	Nombre	Descripción
RF21	Autenticación de usuarios	Todos los usuarios acceden al sistema mediante login y password. El sistema valida las credenciales y determina el tipo de usuario para controlar los permisos disponibles.
RF22	Responsabilidades operativas del mesero	El mesero atiende las mesas asignadas durante su turno y es el responsable de entregar físicamente los juegos prestados a los clientes. El sistema registra qué mesero estuvo asociado a cada préstamo.
RF23	Programa de fidelidad	Cada venta genera automáticamente puntos equivalentes al 1% del total pagado. Los puntos se acumulan en el saldo del usuario y pueden usarse como descuento en compras futuras (1 punto = 1 peso).

6. Diagramas de secuencia

Los diagramas de secuencia ilustran las colaboraciones más complejas del sistema — aquellas con mayor número de validaciones encadenadas. Se eligieron las funcionalidades que mejor ejemplifican cómo interactúan los distintos componentes del diseño.

6.1 Diagrama 1

Funcionalidad más compleja del sistema. Encadena hasta 7 validaciones antes de confirmar el préstamo. Aplica para clientes (con mesa) y empleados (fuera de turno).

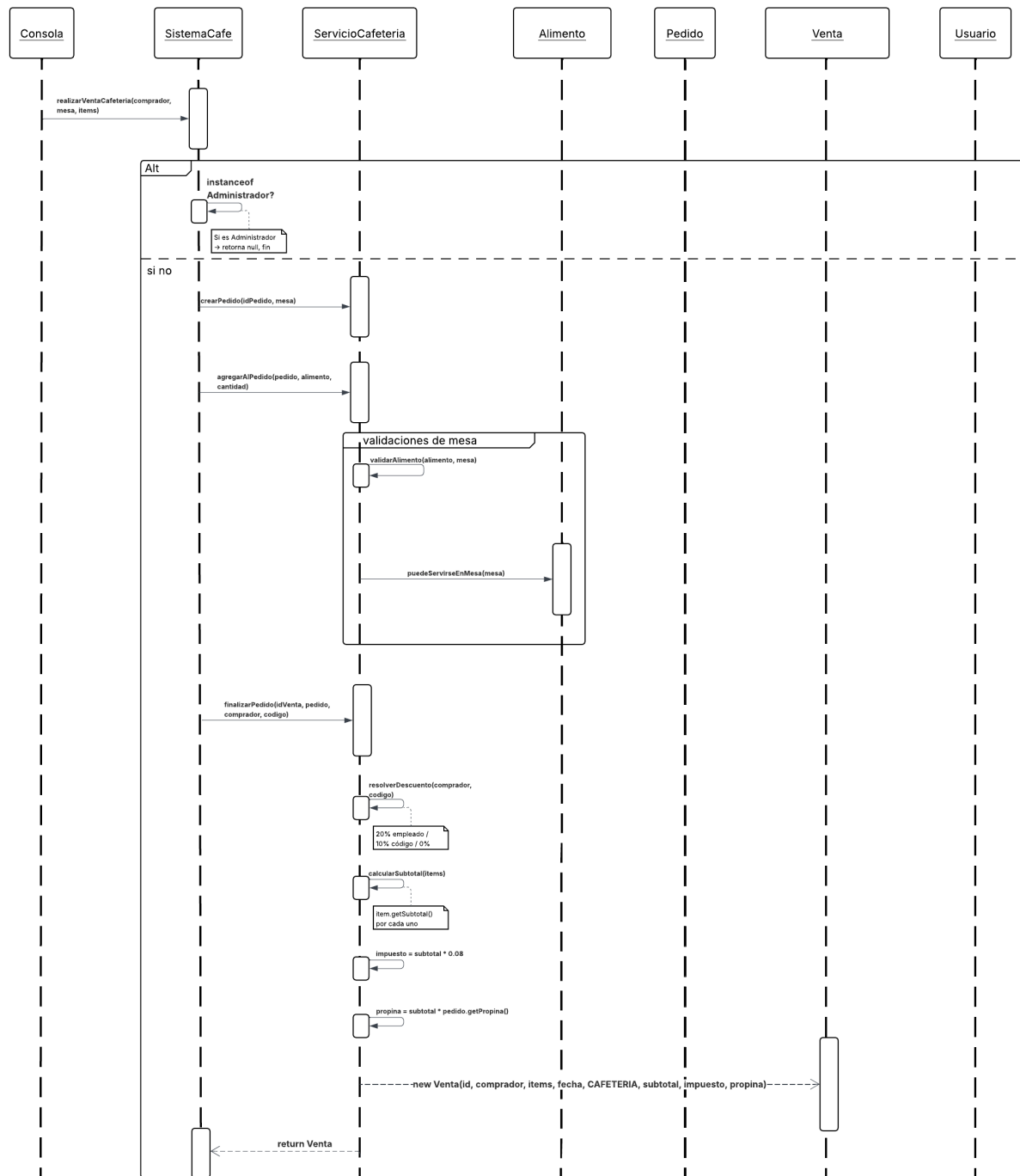


El diagrama empieza en Consola porque alguien real tiene que iniciar la acción, y todo pasa primero por SistemaCafe porque es el portero del sistema, nadie habla directamente con los servicios internos. Lo primero que hace SistemaCafe es verificar si quien pide es un administrador, y si lo es corta inmediatamente porque el enunciado lo prohíbe explícitamente, de ahí el fragmento alt que representa ese if/else. Si pasa esa barrera, el flujo llega a ServicioPrestamo que se hace un checklist interno antes

de salir a preguntar cosas, y las preguntas van en orden de lo más barato a lo más costoso: primero si el empleado está en turno porque es una consulta local, luego si el juego es apto para la edad y el número de personas preguntándole directamente a JuegoDeMesa porque ya tiene esos datos, luego la validación cruzada de bebida caliente con juego de Acción que requiere consultar tanto a JuegoDeMesa como a Mesa porque cada uno sabe lo suyo, y de último se consulta InventarioPrestamo para ver si hay copia disponible porque es la consulta más pesada. Si todo pasa, se verifica el tema del juego difícil pero sin bloquear el préstamo si no hay mesero, solo advirtiéndolo, porque el enunciado lo permite así. Finalmente se crea el Prestamo, y en ese momento el constructor automáticamente marca la copia como no disponible, y se le notifica a la Mesa para que sepa qué juegos tiene asignados y pueda bloquear un tercer préstamo si alguien lo intenta.

6.2 Diagrama 2

Verifica la capacidad máxima del café antes de asignar una mesa al cliente.



El diagrama empieza en Consola porque alguien tiene que iniciar la acción, y todo pasa primero por SistemaCafe porque es el punto de entrada del sistema y el que controla el flujo. Lo primero que hace es verificar si el comprador es administrador, y si lo es, corta inmediatamente retornando null, por eso se usa un fragmento `alt` para representar ese caso.

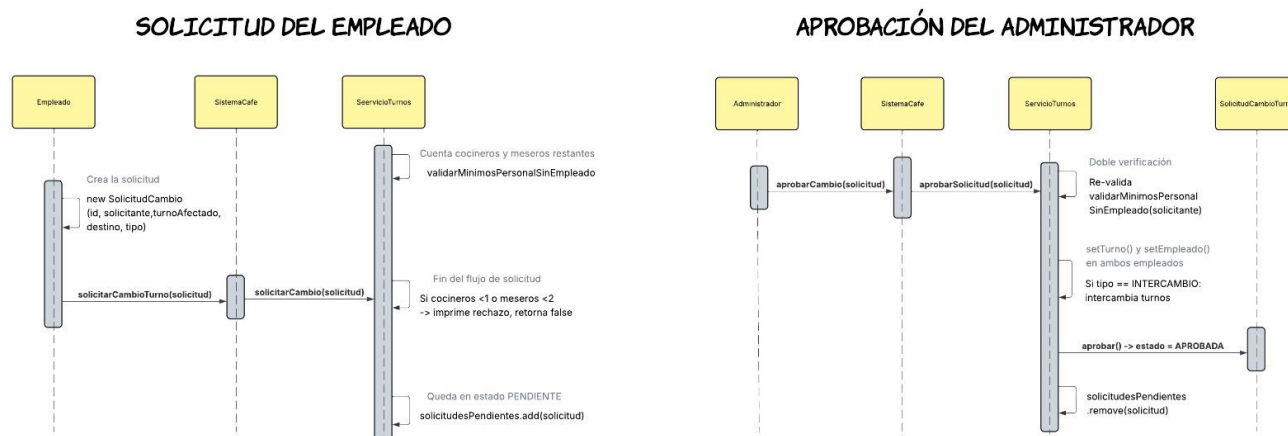
Si pasa esa validación, el flujo continúa hacia ServicioCafeteria, que es el encargado de toda la lógica de la venta. Primero crea el pedido asociado a la mesa y luego agrega los ítems. Después se validan los alimentos consultando directamente a Alimento si pueden servirse en esa mesa. Estas validaciones se agrupan en un `opt` porque funcionan como un bloque lógico dentro del flujo sin cambiar su estructura general.

Una vez validado todo, se finaliza el pedido y se hacen los cálculos. Primero se resuelve el descuento dependiendo del comprador o de un código, luego se calcula el subtotal sumando el valor de cada ítem, y a partir de ahí se calculan el impuesto y la propina. Estos últimos se modelan como self messages porque son cálculos internos del ServicioCafeteria y no necesitan interactuar con otras clases.

Finalmente, con todos los valores listos, se crea la venta, se actualizan los puntos del usuario según la compra y se retorna la venta a SistemaCafe, cerrando el proceso de forma ordenada y centralizada.

6.3 Diagrama 3

Incluye validación de cada alimento, cálculo de impuesto al consumo 8%, propina y acumulación de puntos.



El diagrama empieza en Consola porque alguien tiene que iniciar la acción, y todo pasa primero por SistemaCafe porque es el punto de entrada del sistema y el que controla el flujo. Lo primero que hace es verificar si el comprador es administrador, y si lo es, corta inmediatamente retornando null, por eso se usa un fragmento alt para representar ese caso.

Si pasa esa validación, el flujo continúa hacia ServicioCafeteria, que es el encargado de toda la lógica de la venta. Primero crea el pedido asociado a la mesa y luego agrega los ítems. Cada ítem es validado consultando directamente al objeto Alimento (polimórfico: Bebida o Pastelería) si puede servirse en esa mesa. Las validaciones de Bebida verifican si hay menores (para el alcohol) y si hay juego de Acción activo (para bebidas calientes); la Pastelería informa los alérgenos si los tiene. Estas validaciones se agrupan en un bloque `opt` porque son condiciones de fallo que pueden no ejecutarse.

Una vez validado todo, se finaliza el pedido. Primero se resuelve el descuento (20% si es Empleado, 10% si hay código válido, 0% en otro caso), luego se calcula el subtotal sumando el valor de cada ítem con el descuento aplicado, y a partir de ahí se calculan el impuesto al consumo (8% sobre el subtotal) y la propina (porcentaje configurado en el Pedido, por defecto 10% sobre el subtotal). El orden importa: la propina se calcula sobre el subtotal antes de impuestos para que el cliente no 'propine' sobre el impuesto del gobierno. Estos cálculos se modelan como self messages porque son internos de ServicioCafeteria.

Finalmente, con todos los valores listos, se crea la Venta, se acumulan los puntos de fidelidad (1% del total) en el usuario comprador y se retorna la Venta a SistemaCafe, cerrando el proceso de forma ordenada.

7. Roles, responsabilidades y colaboraciones

Esta sección documenta el diseño de responsabilidades siguiendo la metodología Responsibility-Driven Design estudiada en el curso. Para cada componente principal se define: qué sabe, qué hace, qué decide y con quién colabora, basado en nuestro diagrama y en el código implementado.

7.1 Estilo de control del sistema

El sistema usa un estilo de control delegado, que fue la decisión más importante de arquitectura:

Nivel	Responsabilidad en la arquitectura
Coordinador (SistemaCafe)	Recibe todas las peticiones externas. Delega a los servicios. Valida permisos (el Admin no puede comprar ni pedir prestado, y ningún usuario puede acceder a

	funciones de torneo sin pasar por aquí). A partir del Proyecto 2, también coordina el ciclo de vida de los torneos a través de ServicioTorneo.
Servicios (ServicioPrestamo, etc.)	Tienen la inteligencia del sistema. Saben cómo ejecutar cada operación compleja, encadenar validaciones y coordinar entre objetos del modelo. No guardan estado propio.
Modelos (JuegoDeMesa, Mesa, etc.)	Tienen validaciones de sí mismos. JuegoDeMesa sabe si acepta N jugadores; Bebida sabe si puede servirse en una mesa. No conocen a los servicios ni a SistemaCafe.

Esta distribución evita el estilo centralizado (donde SistemaCafe haría todo y se volvería inmanejable) y un estilo más “disperso” (donde sería difícil encontrar dónde está cada lógica).

7.2 Responsabilidades de los componentes principales

En este caso, se definen las responsabilidades de los componentes principales como los que consideramos importantes para los requerimientos funcionales y de servicios principales.

SistemaCafe Coordinator	
Sabe / conoce	El estado como los usuarios, mesas, inventarios, ventas, sugerencias, etc. Instancias en general de todos los servicios.
Hace	Autentica usuarios, Registra y libera mesas. Delega prestamos, ventas y turnos a los servicios correspondientes. Agrega usuarios/mesas al sistema.
Decide	Si el usuario tiene permiso para operación. A qué servicio delegar cada operación
Colabora con	Todos los servicios del paquete “Servicios”.

ServicioPrestamo Service Provider	
Sabe / conoce	Validar una solicitud de préstamo encadenando con todas las reglas de prestamos correspondientes. Historial de préstamos. Lista de meseros capacitados.
Hace	Validar la cadena completa con el proceso y condiciones para prestar los juegos. Crea el préstamo. Gestiona la devolución.
Decide	Si el préstamo es optimo y valido tras todas las condiciones y restricciones. Que mesero esta capacitado para el juego.
Colabora con	Colabora con InventarioPrestamo, JuegoDeMesa, Mesa y Prestamo, en diferentes métodos.

ServicioVentas Service Provider	
Sabe / conoce	El inventario de las ventas. El historial de las ventas. Los ítems temporales y descuentos en progreso.
Hace	Gestiona el ciclo de la venta de un ítem: Crear, agregar, aplicar descuento y finalizar la venta. Adicionalmente, descuenta stock del inventario.
Decide	Que descuento aplica. Como calcular el total. Cuando descontar el stock.
Colabora con	InventarioVenta, ItemVenta, Venta, Usuario. Todo esto con algunos métodos como vender, crear registros, agregar puntos de fidelidad o agregar compras.

ServicioCafeteria Service Provider	
Sabe / conoce	Valida el menú de alimentos. Historial de ventas. Las constantes de descuento para cada caso.

Hace	Crear pedidos. Validar cada alimento antes de agregarlo. Finalizar pedido acumulando descuento, impuesto o propina.
Decide	Si un alimento puede servirse en una mesa. Qué descuento aplica según el tipo de comprador.
Colabora con	Alimento, Mesa y Venta. Para delegar validaciones específicas, la composición del grupo y creación de registros.

ServicioTurnosService

Provider

Sabe / conoce	Lista de empleados. Lista de turnos. Lista de solicitudes pendientes. Identificación única según empleado (1 para cocinero, 2 para mesero)
Hace	Crear y eliminar turnos. Registrar solicitudes de cambio. Aprobar o rechazar solicitudes de intercambio de turno entre empleados.
Decide	Si el intercambio cumple los mínimos de personal. Si el intercambio de turno es válido.
Colabora con	Turno, Empleado, SolicitudCambioTurno, en métodos como seleccionar empleados, mirar si esta en turno el empleado, y en aprobar o rechazar solicitudes.

JuegoDeMesa

Information Holder

Sabe / conoce	Sus propias restricciones como numero de jugadores, edad mínima, categoría, si es difícil o no, etc.
Hace	Responder a consultas de validación sobre sí mismo.
Decide	No toma decisiones. Solo responde si es apto para personas, si es apto por dificultad si es compatible con bebidas calientes o si es difícil.
Colabora con	Nadie. Es consultado por ServicioPrestamo y Mesa pero no llama a nadie.

Mesa

Information Holder

Sabe / conoce	Su composición actual como personas, niños, jóvenes. Sus prestamos activos. Su pedido actual (como por ejemplo el de las bebidas).
Hace	Gestiona su ocupación y liberación. Validar y registrar nuevos prestamos. Responder sobre su estado para que otras clases funcionen y tomen decisiones.
Decide	No toma decisiones del sistema. Responde sobre sí misma como por ejemplo en niños y jóvene, juegos de acción, bebidas calientes o cantidad de juegos prestados.
Colabora con	Colabora con JuegoDeMesa, Prestamo y ItemVenta/Pedido para validar prestamos, liberar mesas y para confirmar las bebidas en la mesa.

ServicioTorneo

— Service Provider

Sabe / conoce	Lista de torneos creados. Las reglas de cupo (general, fanáticos, regulares). El inventario de préstamo (para validar copias). Los límites de inscripción (máx. 3 participantes por usuario).
Hace	Crear torneos amistosos y competitivos con validación de copias. Gestionar el ciclo de inscripción: validar, distribuir cupos, calcular monto. Registrar ganadores y distribuir premios (bono de descuento o premio metálico).
Decide	Si hay suficientes copias para el cupo del torneo. Cómo distribuir cupos entre fanáticos y regulares. Si el empleado puede inscribirse (verifica turno). Si el ganador puede recibir el premio metálico.
Colabora con	torneos, Inscripcion, InventarioPrestamo, Empleado, Cliente, Usuario — para validar, crear y registrar inscripciones y premios.

8. Decisiones de diseño

A continuación, se documentan las decisiones de diseño más importantes, las alternativas consideradas y los “costos de oportunidad” que asumimos. Algunas de estas decisiones ya se encontraban en nuestro primer diagrama del análisis del mundo en la entrega 1, sin embargo, consideramos importante justificarlas ya que ayudan a entender el flujo de nuestro trabajo.

Decisión 1 — El UML inicial como brújula de nuestra entrega.	
Decisión tomada	Antes de empezar a implementar en código, decidimos que era mejor empezar contruyendo el diagrama de clases principal en base al análisis del mundo entregado en la entrega 1, modelando todas las entidades del dominio con relaciones, herencias, atributos y métodos.
Alternativa considerada	Empezar a programar directamente, diseñando las clases sobre la marcha a medida que aparecían las necesidades.
Por qué se eligió	El UML inicial fue, honestamente, la decisión mas valiosa del proyecto. No porque este fuera perfecto (de hecho, se cambio muchísimo incluso después del código), sino porque representó una pieza importante para comenzar el proyecto. Es decir, nos obligo a ponernos serios con el dominio antes de entrar a eclipse. Nos ayudpo a distinguir cuando algo era una clase y cuando era solo un atributo. Como por ejemplo, separar SolicitudCambioTurno de Turno ya que nos dimos cuenta que el proceso de solicitar es completamente distinto al concepto de tener un turno asignado. El diagrama nos dio un lenguaje común para todos nosotros, y sobre cada vez que había duda de donde cumplir el requerimiento funcional o como implementar algo, solo teníamos que mirarlo y orientarnos. Aprendimos mas sobre diseño construyendo ese diagrama en primer lugar de la implementación que sobre la implementación en sí misma.
Costo de oportunidad asumido	El diagrama inicial no fue la versión final. Cosas como algunas clases o atributos cambiaron bastante durante la implementación. Esto nos pareció normal y esperado en un proceso de diseño iterativo. Quizá fue algo molesto tener que actualizarlo siempre, y quizá esto puede que haya hecho que se nos hayan quedado algunas cosas pero sentimos que valió completamente la pena.

Decisión 2 — Mesa como validador de prestamos.	
Decisión tomada	Mesa.agregarPrestamo() encadena sus propias validaciones como el limite de juegos, rango de jugadores, restricción de edad y restricciones como bebidas o juegos de acción. Esta lógica también existe en ServicioPrestamo.validarPrestamo()
Alternativa considerada	Dejar toda la validación en ServicioPrestamo sin que Mesa haga ninguna verificación propia.

Por qué se eligió	Mesa es Modelo propio de su propio estado, es decir, es la única que sabe cuantos prestamos tiene, si hay niños, si hay bebida caliente, etc. Colocar al menos parte de esas validaciones en Mesa respeta el principio de información y hace que la clase sea mas “robusta”.
Costo de oportunidad asumido	La lógica de validación esta duplicada ente Mesa.agregarPrestamo() y ServicioPrestamo.validarPrestamo(). Esto crea un riesgo de inconsistencia i una de las dos se actualiza sin actualiza la otra. En el código actual, ServicioPrestamo llama a validarPrestamo() primero y luego a mesa.agregarPrestamo(), lo que significa que las validaciones se ejecutan dos veces (y evita que podamos tener el posible riesgo).

Decisión 3 — Separar JuegoDeMesa de CopiaJuego

Decisión tomada	JuegoDeMesa es el catálogo conceptual. CopiaJuego es la unidad física con su propio estado y disponibilidad.
Alternativa considerada	Una sola clase JuegoDeMesa con campos cantidad:int y disponible:boolean.
Por qué se eligió	El enunciado exige: rastrear cuántas veces se prestó cada unidad, conocer el estado físico individual y que el administrador mueva unidades específicas entre inventarios. Nada de esto es posible sin una clase por unidad física.
Costo de oportunidad asumido	Aumenta el número de clases. Cada préstamo referencia una CopiaJuego específica. El constructor de Pestramo llama a copia.marcarNoDisponible() automáticamente, lo que acopla el constructor con el estado de CopiaJuego.

Decisión 4 — Jerarquía de herencia en dos niveles para usuarios

Decisión tomada	Usuario (abstracta) - Cliente / Empleado (abstracta) / Administrador. Empleado - Mesero / Cocinero.
Alternativa considerada	Una sola clase Usuario con un campo rol:enum y atributos opcionales.
Por qué se eligió	Los tres tipos tienen atributos fundamentalmente distintos: Cliente tiene reservas; Mesero tiene atributos con los juegos; Administrador no puede hacer préstamos. La herencia en dos niveles permite diferenciar este tipo de diferencias entre los usuarios y los métodos que estos pueden ejecutar.
Costo de oportunidad asumido	Una jerarquía de herencia es más rígida que composición. Cambiar el rol de un usuario requeriría crear un nuevo objeto.

Decisión 5 — InventarioPrestamo e InventarioVenta como clases independientes sin jerarquía.

Decisión tomada	InventarioPrestamo usa List<CopiaJuego> e InventarioVenta usa Map<JuegoDeMesa,Integer>. No existe clase base inventario.
Alternativa considerada	Una clase abstracta Iventario con subclases, como estaba planteado en el diagrama inicial de la entrega 1
Por qué se eligió	La estructura de datos es radicalmente diferente, pues préstamo trabaja con copias físicas individuales mientras que venta trabaja con cantidades y precios por título. En la practica, la jerarquía abstracta no aportaba ningun método que ambas compartieran de forma útil. Se prefirió simplicidad sobre la “elegancia” estructural.
Costo de oportunidad asumido	Perdemos la posibilidad de tratar ambos inventarios de forma polimórfica. El administrador tiene que manejarlos como objetos completamente distintos, y ServicioInventario debe conocer ambos tipos concretos.

Decisión 6 — SistemaCafe como coordinador con servicios internos

Decisión tomada	SistemaCafe mantiene todos los servicios en su constructor. Es el único punto de entrada del sistema.
Alternativa considerada	Servicios independientes que se crean fuera del SistemaCafé y se pasan como parámetros.
Por qué se eligió	Simplifica el uso del sistema. Es decir, quien cre un SistemaCafe no necesita saber nada sobre como están organizados los servicios internamente. Centraliza la creación y ganatiza que todos los servicios trabajen con el mismo inventario y las mismas listas.
Costo de oportunidad asumido	El consturctor de SistemaCafe extrae meseros y empleados de la lista de usuarios ANTES de que se hayan agregado usuarios. Las listas quedan “vacías” inicialmente. Esto se compensa con agregarUsuario(), que los agrega a los servicios correctos después, pero es un acoplamiento que puede llegar a ser frágil y generar bugs si el orden de llamadas no es el correcto.

Decisión 7 — Jerarquía de herencia para torneos

Decisión tomada	torneos (abstracta) como clase base con torneoAmistoso y torneoCompetitivo como subclases concretas. Incripcion como clase de detalle independiente que registra la distribución de cupos, el monto y si puede recibir el premio.
Alternativa considerada	Una sola clase Torneo con un campo tipo:enum y atributos opcionales (bonoPremio o tarifaEntrada según el tipo).
Por qué se eligió	El comportamiento es fundamentalmente diferente: amistoso nunca cobra entrada y siempre otorga bono; competitivo siempre cobra (salvo empleados) y calcula el premio como suma de lo pagado. Con

	enum, habría un if/switch en cada método relevante. Con herencia, cada subclase implementa solo lo que le corresponde (esAmistoso(), getTipo(), calcularPremio()) sin condicionales adicionales. El patrón sigue exactamente la misma decisión tomada para Alimento/Bebida/Pasteleria.
Costo de oportunidad asumido	Mayor número de clases. Si surgiera un tercer tipo de torneo, se crearía una nueva subclase (extensible). Sin embargo, el switch en ServicioTorneo sigue siendo necesario para la lógica de pago (torneoCompetitivo instanceof), lo que genera un pequeño acoplamiento con el tipo concreto.

Decisión 8 — Inscripcion como clase “diferente” (no atributos en torneos)

Decisión tomada	Inscripcion es una clase separada que registra cuposFanaticosUsados, cuposRegularesUsados, puedeRecibirPremioMetalico y montoPagado. No son atributos de torneos.
Alternativa considerada	Guardar en torneos mapas de usuario->cupos o listas de usuarios inscritos con datos adicionales como campos directos.
Por qué se eligió	Cada inscripción tiene datos propios que no pertenecen al torneo sino a la relación usuario-torneo: cuántos cupos tomó ese usuario, de qué tipo, cuánto pagó. Separarlo en Inscripcion respeta el principio de información experta y facilita preguntar '¿puede el ganador recibir el premio?' sin buscar en el torneo.
Costo de oportunidad asumido	Para buscar la inscripción de un usuario hay que iterar la lista de Inscripcion del torneo (buscarInscripcion()). No es posible hacer un acceso directo por clave.

9. APIs de los servicios

En el contexto de este proyecto, se describirán las APIs de los servicios, dado que son el conjunto de métodos públicos para que las otras clases o consola principal usen para los requerimientos funcionales del proyecto. En este caso, solo se realizan la de los servicios principales ya que estas son las que el resto del sistema “puede pedirle” o pueden utilizar.

Describir la APIs de toda nuestra implementación puede sonar redundante, ya que podemos ver un poco de sus estructuras en el diagrama de clases principal y en los diagramas de secuencia.

9.1 SistemaCafe

SistemaCafe			
Método	Parámetros	Retorna	Qué valida / comportamiento
login()	String login, String pwd	Usuario	Itera usuarios llamando autenticar(). Retorna null si no hay coincidencia.
registrarMesa()	int numPersonas, int numNinos, int numJovenes, Cliente cliente	Mesa	Verifica capacidad (suma personas de mesas ocupadas). Llama mesa.ocupar() y cliente.setMesaActual(). Retorna null si lleno o sin mesas.

prestarJuego()	Usuario usuario, JuegoDeMesa juego, Mesa mesa	Prestamo	Bloquea Admin. Genera id con timestamp. Delega a servicioPrestamo. Retorna null si validación falla.
devolverJuego()	Prestamo prestamo	void	Delega a servicioPrestamo.devolverJuego().
realizarVentaTienda()	Usuario comprador, List<ItemVenta> items	Venta	Bloquea Admin. Ciclo: crearVenta-agregarItems-aplicarDescuento-finalizarVenta. Agrega a ventas si no null.
realizarVentaCafeteria()	Usuario comprador, Mesa mesa, List<ItemVenta> items	Venta	Bloquea Admin. Crea pedido, agrega items validados, finaliza con código de descuento del empleado si aplica.
solicitarCambioTurno()	SolicitudCambioTurno solicitud	void	Delega a servicioTurnos.solicitarCambio().
aprobarCambio()	SolicitudCambioTurno solicitud	void	Delega a servicioTurnos.aprobarSolicitud().
rechazarCambio()	SolicitudCambioTurno solicitud	void	Delega a servicioTurnos.rechazarSolicitud().
liberarMesa()	Cliente cliente	void	Obtiene mesaActual del cliente. Llama mesa.liberar() y cliente.setMesaActual(null).
agregarAlimento()	Alimento a	void	Delega a servicioCafeteria.agregarAlMenu().
aprobarSugerencia()	Sugerencia sugerencia	void	Llama sugerencia.aprobar(). El admin decide si además agrega el alimento al menú.
generarInforme()	TipoVenta tipo, Date desde, Date hasta	List<Venta>	Delega a servicioReportes.generarInformeVentas().
agregarUsuario()	Usuario usuario	void	Agrega a lista. Si es Empleado: lo agrega a servicioTurnos. Si es Mesero: lo agrega a servicioPrestamo.
agregarMesa()	Mesa mesa	void	Delega a mesas.add()

9.2 ServicioPrestamo

ServicioPrestamo			
Método	Parámetros	Retorna	Qué valida / comportamiento
prestarJuego()	String idPrestamo, Usuario usuario, JuegoDeMesa juego, Mesa mesa, List<Mesa> todasLasMesas	Prestamo	Llama validarPrestamo(). Si false: retorna null. Busca copia disponible. Imprime Advertencia si juego difícil sin mesero. Crea Prestamo y lo agrega al historial.

devolverJuego()	Prestamo prestamo	void	Llama prestamo.devolver(). Si mesa!=null: remueve de prestamosActivos.
validarPrestamo()	Usuario usuario, JuegoDeMesa juego, Mesa mesa, List<Mesa> todasLasMesas	boolean	Admin→false. Empleado: verifica estaEnTurno()+hayClientes. Cliente: verifica mesa, ≤2 activos, jugadores, edad, accion+caliente.
buscarMeseroCapacitado()	JuegoDeMesa juego, List<Mesero> meseros	Mesero	Retorna primer Mesero con conoceJuego(j)==true && estaEnTurno()==true. Null si ninguno.
getPrestamosActivos()	-	List<Prestamo>	Filtra historialPrestamos donde isActive()==true.
getHistorialPrestamos()	-	HistorialPrestamos	Retorna atributo asignado
Getmeseros()	-	Meseros	Retorna atributo asignado

9.3 ServicioVentas

ServicioVentas			
Método	Parámetros	Retorna	Qué valida / comportamiento
crearVenta()	String idVenta, Usuario comprador, TipoVenta tipo	null	Inicializa mapas temporales itemsEnProgreso y descuentosEnProgreso para el id. RETORNA NULL - la venta real se crea en finalizarVenta().
agregarItem()	String idVenta, ItemVenta item	void	Agrega item a itemsEnProgreso.get(idVenta) si existe el id.
aplicarDescuento()	String idVenta, String codigoDescuento, Usuario comprador	void	resolverDescuento(): 20% si Empleado, 10% si código no vacío, 0 en otro caso. Guarda en descuentosEnProgreso.
finalizarVenta()	String idVenta, Usuario comprador, TipoVenta tipo	Venta	Aplica descuento a items de juego. Llama inventarioVenta.vender() por cada juego. Calcula subtotal+IVA19%. Crea Venta. Acumula puntos. Limpia mapas temporales.
calcularTotal()	List<ItemVenta> items	double	Suma item.getSubtotal() para cada item.

resolverDescuento()	Usuario comprador, String codigoDescuento	double	Empleado-0.20, código no vacío-0.10, ninguno-0.0.
getHistorialVentas()	-	HistorialVentas	Retorna atributo asignado

9.4 ServicioCafeteria

ServicioCafeteria			
Método	Parámetros	Retorna	Qué valida / comportamiento
validarAlimento()	Alimento alimento, Mesa mesa	boolean	Delega a alimento.puedeServirseEnMesa(mesa). Retorna false si no puede servirse.
crearPedido()	String idPedido, Mesa mesa	Pedido	Crea nuevo Pedido y llama mesa.setPedidoActual(pedido).
agregarAlPedido()	Pedido pedido, Alimento alimento, int cantidad	void	Valida alimento. Si rechazado: imprime mensaje. Si válido: pedido.agregarItem() con descuento=0 (se aplica al finalizar).
finalizarPedido()	String idVenta, Pedido pedido, Usuario comprador, String codigoDescuento	Venta	resolverDescuento(). Recrea items con descuento. Calcula subtotal, impuesto 8%, propina=subtotal×propina%. Crea Venta CAFETERIA. Acumula puntos.
agregarAlMenu()	Alimento alimento	void	Agrega al List<Alimento> menu.
resolverDescuento()	Usuario comprador, String codigoDescuento	DescuentoEmpleado o DescuentoCodigo	Calcula el descuento según corresponda utilizado en finalizarPedido()
getmenu()	-	Menu	Retorna atributo asignado
getHistorialVentas	-	HistorialVentas	Retorna atributo asignado

9.5 ServicioTurnos

ServicioTurnos			
Método	Parámetros	Retorna	Qué valida / comportamiento
crearTurno()	String idturno, Empleado empleado, String dia	Turno	Crea Turno, llama empleado.setTurno(turno), agrega a turnos.

eliminarTurno()	Turno turno	void	Llama <code>t.getEmpleado().setTurno(null)</code> . Remueve de turnos.
solicitarCambio()	SolicitudCambio Turno solicitud	boolean	<code>validarMinimosPersonalSinEmpleado</code> (solicitante). Si falla: false. Si pasa: agrega a <code>solicitudesPendientes</code> .
aprobarSolicitud()	SolicitudCambio Turno solicitud	boolean	Re-valida mínimos. Si INTERCAMBIO: intercambia <code>setTurno()</code> y <code>setEmpleado()</code> . Llama <code>solicitud.aprobar()</code> . Remueve de pendientes.
rechazarSolicitud()	SolicitudCambio Turno solicitud	void	Llama <code>solicitud.rechazar()</code> . Remueve de pendientes.
validarMinimosPersonal()	String día	boolean	Cuenta por tipo en turnos del día. Retorna <code>cocineros ≥ 1 && meseros ≥ 2</code> .
validarMinimosPersonalSinEmpleado()	Empleado empleadoSaliente	boolean	Valida el personal saliente por medio del mínimo de empleados que se necesitan.
getSolicitudesPendientes()	-	SolicitudesPendientes	Retorna atributo asignado
getTurnos()	-	Turnos	Retorna atributo asignado
getEmpleados()	-	Empleados	Retorna atributo asignado

9.6 ServicioInventario

ServicioInventario			
Método	Parámetros	Retorna	Qué valida / comportamiento
agregarCopiaAPrestamo()	CopiaJuego copia	void	<code>inventarioPrestamo.agregarCopia(c)</code> .
repararCopia()	CopiaJuego copiaAReemplazar, JuegoDeMesa juego	boolean	Si <code>!inventarioVenta.hayStock(j)</code> : false. Descuenta 1 del <code>inventarioVenta</code> . Llama <code>cRota.actualizarEstado("Nuevo")</code> y <code>cRota.marcarDisponible()</code> .
marcarComoRobada()	CopiaJuego copia	void	<code>c.actualizarEstado("Desaparecido")</code> . <code>c.marcarNoDisponible()</code> .
reabastecerVenta()	JuegoDeMesa juego, int cantidad, double precio	void	<code>inventarioVenta.reabastecer(j, cantidad, precio)</code> .
moverDeVentaAPrestamo()	JuegoDeMesa juego, String idNuevaCopia	boolean	Si <code>!hayStock</code> : false. <code>vender(j,1)</code> . Crea <code>CopiaJuego</code> nueva con estado "Nuevo". <code>agregarCopia()</code> .

actualizarEstadoCopia()	CopiaJuego copia, String nuevoEstado	void	c.actualizarEstado(estado).
reabastecerPrestamo()	CopiaJuego copia	void	inventarioPrestamo.agregarCopia(copia)
getCopiasPorJuego()	JuegoDeMesa juego	CopiasJuego	inventarioPrestamo. <u>getCopiasPorJuego(juego)</u>
getInventarioPrestamo()	-	InventarioPrestamo	Retorna atributo asignado
getInventarioVenta()	-	IventarioVenta	Retorna atributo asignado

9.7 ServicioReportes

ServicioReportes			
Método	Parámetros	Retorna	Qué valida / comportamiento
generarInformeVentas()	TipoVenta tipo, Date desde, Date hasta	Resultado	Filtra la lista (JUEGO→ventasJuegos, CAFETERIA→ventasCafeteria) por rango de fechas.
ventasPorDia()	Date fecha	Resultado	Combina ambas listas. Filtra por mismoDia() comparando YEAR y DAY_OF_YEAR.
ventasPorSemana()	int semana	Resultado	Combina ambas listas. Filtra por Calendar.WEEK_OF_YEAR.
ventasPorMes()	int mes	Resultado	Combina ambas listas. Filtra por Calendar.MONTH (0-indexado).
getTotalSubtotal()	List<Venta> ventas	double	Suma v.getSubtotal() para cada venta.
getTotalImpuestos()	List<Venta> ventas	double	Suma v.getImpuesto() para cada venta.
getTotalPropinas()	List<Venta> ventas	double	Suma v.getPropina() para cada venta.
mismoDia()	Date a, Date b	boolean	Método de apoyo para verificar si tenemos un mismo día, por medio de funciones auxiliares como Calendar

9.8 ServicioTorneo

ServicioTorneo			
Método	Parámetros	Retorna	Qué valida / comportamiento
crearTorneoAmistoso() ()	String idTorneo, String nombre, JuegoDeMesa juego, String diaSemana, int cupoMaximo, double bonoPremio	torneos	validarCupoConCopias(): copiasDisponibles × maxJugadores >= cupoMaximo. Si falla: retorna null. Crea torneoAmistoso y lo agrega a la lista.
crearTorneoCompetitivo() ()	String idTorneo, String nombre, JuegoDeMesa juego, String diaSemana, int cupoMaximo, double tarifaEntrada	torneos	Misma validación de copias. Crea torneoCompetitivo y lo agrega a la lista.
inscribirse()	torneos torneo, Usuario usuario, int numParticipantes	boolean	Valida: 1<=num<=3, no duplicado, cupos totales, turno del empleado. Distribuye cupos fanáticos/regulares. En competitivo: empleado monto=0 y puedeRecibir=false; cliente monto=tarifa×num. Crea y registra Inscripcion.
desinscribirse()	torneos torneo, Usuario usuario	boolean	torneo.eliminarInscripcion(usuario). Retorna false si no estaba inscrito.
registrarGanador()	torneos torneo, Usuario ganador	Inscripcion	Busca inscripcion del ganador. Si amistoso: guardarBonoPremioAmistoso() en el usuario. Si competitivo: informa premio o advierte que empleado no lo recibe. Retorna la Inscripcion del ganador.
empleadoCubreTurnoEseDia() ()	Empleado empleado, String diaTorneo	boolean	Retorna true si empleado.estaEnTurno() && empleado.getTurno().getDiaSemana().e qualsIgnoreCase(diaTorneo).
buscarTorneo()	String idTorneo	torneos	Itera la lista buscando por idTorneo. Retorna null si no existe.
getTorneos()	—	List<torneos>	Retorna la lista completa de torneos.

10. Diseño de la persistencia

En este proyecto, la persistencia se diseñó con el objetivo de almacenar y recuperar toda la información del sistema de forma estructurada, manteniendo la coherencia entre todas las clases del mundo implementado.

A diferencia de un enfoque con bases de datos o archivos JSON, se optó por el uso de archivos de texto plano (.txt) organizados dentro de una carpeta datos/ ubicada fuera del directorio src/. Esta decisión se tomó principalmente por simplicidad de implementación, control directo sobre el formato de los datos y facilidad para depuración durante el desarrollo, adicionalmente, creemos que de esta forma es más fácil el manejo de las pruebas y testeos del sistema. En caso de utilizar una base de datos o archivos tipo JSON las persistencias tendrían que ser adaptadas.

El sistema implementa una arquitectura donde existe una clase de persistencia por cada conjunto principal de información que nosotros consideramos importante orara mantener información con el tiempo, lo cual permite separar responsabilidades y mantener el código organizado. Además, existe una clase central, PersistenciaCafe, que se encarga de coordinar todo el proceso de guardado y carga, respetando el orden de dependencias entre entidades, tal como lo hacen las clases principales con el sistema principal Sistema.

10.1 Archivos de datos

A continuación, se describen los archivos utilizados en el sistema, junto con su contenido y propósito:

Archivo	Contenido y propósito
datos_prueba/cafe.txt	Almacena la configuración general del sistema, específicamente la capacidad máxima del café. Es el primer archivo que se lee, ya que define parámetros necesarios para inicializar el sistema.
datos_prueba//mesas.txt	Contiene los identificadores de las mesas. No se guarda su estado directamente, ya que este se reconstruye a partir de los préstamos activos.
datos_prueba//juegos.txt	Guarda el catálogo de juegos (JuegoDeMesa) con todos sus atributos (nombre, número de jugadores, categoría, etc.). Sirve como base para otras entidades que dependen de los juegos.
datos_prueba//copias_prestamo.txt	Contiene las copias físicas de los juegos disponibles para préstamo (CopiaJuego), incluyendo su estado y disponibilidad.
datos_prueba//inventario_venta.txt	Almacena los juegos disponibles para la venta junto con su cantidad y precio.
datos_prueba//usuarios.txt	Contiene todos los usuarios del sistema. Se utiliza un identificador de tipo (CLIENTE, MESERO, COCINERO, ADMIN) para reconstruir la subclase correspondiente al cargar.
datos_prueba//menu.txt	Guarda los productos del menú (Alimento), diferenciando entre Bebida y Pastelería mediante un campo de tipo.
datos_prueba//turnos.txt	Registra los turnos asignados a los empleados, incluyendo el día y el empleado correspondiente.
datos_prueba//solicitudes.txt	Contiene las solicitudes de cambio de turno, junto con su estado (pendiente, aprobada o rechazada).

datos_prueba//prestamos.txt	Guarda el historial completo de préstamos. No se eliminan registros, ya que son necesarios para mantener consistencia de nuestra implementación.
datos_prueba//ventas.txt	Contiene el historial completo de ventas, incluyendo tanto productos de tienda como de cafetería, junto con sus ítems asociados.
datos_prueba//sugerencias.txt	Almacena las sugerencias realizadas por los empleados, incluyendo su estado y relación con el usuario que la propuso.

10.2 Justificación del diseño de persistencia.

El sistema cuenta con múltiples clases de persistencia (PersistenciaUsuarios, PersistenciaJuegos, PersistenciaCopias, etc.), cada una encargada de una parte específica del modelo. Esta decisión se tomó por varias razones:

En primer lugar, separar la persistencia por entidades permite una mejor organización del código. Cada clase tiene una responsabilidad clara (por ejemplo, PersistenciaUsuarios solo maneja usuarios), lo que facilita el mantenimiento y futuras modificaciones, tal como se explicó en el curso.

En segundo lugar, este enfoque permite manejar correctamente las dependencias entre clases. Por ejemplo, los préstamos dependen de usuarios, copias y mesas, por lo que deben cargarse después de estas. La clase PersistenciaCafe coordina este proceso, asegurando que los datos se reconstruyan en el orden correcto, tal como lo explicaremos en otra sección, la cual explica la importancia de el orden en el SistemaCafe, que está relacionado directamente con PersistenciaCafé.

Finalmente, se decidió no eliminar registros en archivos como prestamos.txt y ventas.txt, ya que funcionan como historiales del sistema, lo cual es importante para reportes requeridos en los requerimientos funcionales establecidos.

11. Algoritmos críticos

Esta sección muestra el código real de los algoritmos que nosotros consideramos más complejos, tal como fueron implementados.

11.1 Mesa.agregarPrestamo() — la clase más compleja

Mesa es el componente más complejo del sistema porque encadena múltiples restricciones del enunciado en un solo método. Cualquier error aquí afecta directamente la experiencia del usuario. Pero en especial el método agregarPrestamo() ya que es el que más restricciones y validaciones debe revisar.

Adicionalmente, el código llama a varias funciones auxiliares que le ayudan a la validación del sistema, que se pueden consultar en el código principal en la clase Mesa.

```
public boolean agregarPrestamo(Prestamo prestamo) {  
  
    if (prestamosActivos.size() >= 2) {  
        System.out.println("La mesa ya tiene 2 juegos prestados.");  
        return false;  
    }  
  
    JuegoDeMesa juego = prestamo.getCopia().getJuego();  
  
    if (!juego.esAptoParaPersonas(numeroPersonas)) {  
        System.out.println("El juego no soporta el número de personas.");  
        return false;  
    }  
  
    if (!juego.esAptoParaEdad(hayNinos(), hayJovenes())) {  
        System.out.println("El juego no es apto para la edad de los comensales.");  
        return false;  
    }  
  
    if (tieneBebidaCaliente() && juego.getCategoria().equals("Accion")) {  
        System.out.println("No se puede pedir juego de Acción con bebida caliente.");  
        return false;  
    }  
  
    prestamosActivos.add(prestamo);  
    return true;  
}
```

Primero, verifica que la mesa no tenga ya 2 juegos activos (esta es una de las restricciones más importantes de evaluar, así que va primero). Luego obtiene el juego de la copia y verifica que acepte el número de personas sentadas (esAptoParaPersonas). Después verifica la restricción de edad: si hay niños menores de 5 años o jóvenes menores de 18, el juego debe ser apto para ellos (esAptoParaEdad).

Finalmente, verifica la restricción de bebida: si la mesa ya tiene una bebida caliente en su pedido actual, no puede tener un juego de categoría Acción, y viceversa.

Pero... ¿Porque importa el orden? El orden no es arbitrario. Las primeras verificaciones son las más baratas computacionalmente cuando hablamos de ser un poco más óptimos con nuestros algoritmos (comparar un entero, comparar un booleano). La verificación de bebida caliente es la más costosa porque itera sobre los ítems del pedido actual (tieneBebidaCaliente() recorre la lista), así que se deja para el final. Si algo falla antes, nunca se llega a esa iteración, y por ende, puede llegar a ser más rápido.

El riesgo principal de esta clase: la misma lógica de validación existe duplicada en ServicioPrestamo.validarPrestamo(). Si en el futuro se cambia una regla (por ejemplo, permitir 3 juegos en lugar de 2), hay que cambiarlo en **ambos** lugares. Si se cambia solo en uno, el sistema se vuelve inconsistente, además tantas verificaciones de restricciones pueden llegar a confundir, ya que es difícil perderse.

11.2 ServicioPrestamo.validarPrestamo() — cadena de validaciones

Es el método con la lógica de negocio más extensa. Similar a la anteriormente mencionada. Por ello, también la clasificamos como uno de los más críticos, al tener que revisar también las restricciones principales de nuestro sistema y contexto.

De igual forma, como el algoritmo anterior, este método utiliza varias funciones auxiliares para ayudar con las validaciones, esto se puede consultar en las clases de Mesa, Empleado y JuegoDeMesa.

```
public boolean validarPrestamo(Usuario usuario, JuegoDeMesa juego, Mesa mesa, List<Mesa>
todasLasMesas) {
    if (usuario instanceof Administrador) {
        System.out.println("El administrador no puede pedir juegos prestados.");
        return false;
    }
    if (usuario instanceof Empleado) {
        Empleado empleado = (Empleado) usuario;
        if (empleado.estaEnTurno()) {
            boolean hayClientes = false;
            for (Mesa m : todasLasMesas) {
                if (m.isOcupada()) {
                    hayClientes = true;
                    break;
                }
            }
            if (hayClientes) return false;
        }
        return true;
    }
    // Validaciones para cliente
    if (mesa == null || !mesa.isOcupada()) {
        return false;
    }
}
```



```

    if (mesa.cantidadJuegosPrestados() >= 2) {
        return false;
    }
    if (!juego.esAptoParaPersonas(mesa.getNumeroPersonas())) {
        return false;
    }
    if (!juego.esAptoParaEdad(mesa.hayNinos(), mesa.hayJovenes())) {
        return false;
    }
    if (!juego.esCompatibleConBebidaCaliente() && mesa.tieneBebidaCaliente()) {
        return false;
    }
    return true;
}

```

El método recibe al usuario, el juego solicitado, la mesa y la lista completa de todas las mesas del café. Primero, bloquea al Administrador explícitamente (esta es la única verificación que aplica a todos los tipos de usuario). Luego, actúa según el tipo de actor correspondiente.

Si es un Empleado: verifica si está en turno. Si está en turno, recorre todas las mesas buscando si alguna está ocupada, luego, si hay, aunque sea un cliente en el café, el préstamo se bloquea (según las restricciones del proyecto). Si no hay clientes, el empleado puede pedir, aunque esté en turno. Si el empleado no está en turno, puede pedir siempre sin más restricciones.

Si es un Cliente: la validación es más restrictiva. Verifica que tenga mesa asignada y que esté ocupada, que no tenga ya 2 préstamos activos, que el juego sea apto para el número de personas, que sea apto para las edades presentes, y finalmente la compatibilidad con bebidas calientes según las restricciones del proyecto.

Los empleados tienen un conjunto de restricciones completamente diferente al de los clientes. El enunciado dice explícitamente que los empleados no necesitan mesa, pero que deben estar fuera de turno. Si se pusieran todas las validaciones juntas sin bifurcar, un empleado necesitaría mesa (que no tiene) y un cliente necesitaría verificar si hay otros clientes en el café (que no tiene sentido para él).

11.3 ServicioCafeteria.finalizarPedido() — cálculo completo de venta

Concentra el cálculo de descuento, impuesto al consumo y propina. Es el equivalente del “cierre de cuenta” en el café, adicionalmente, gracias a este es que podemos en parte agregar al historial de compra, por lo que, este nos pareció también muy importante y difícil de implementar debido a las restricciones de impuestos, propinas y puntos de fidelización.

Igualmente, este método usa funciones auxiliares para ayudar con el cálculo de los ítems de venta y puntos de fidelidad, que se pueden consultar en las clases Pedido, Venta e Itemventa.

```

public Venta finalizarPedido(String idVenta, Pedido pedido, Usuario comprador, String
codigoDescuento) {
    double descuento = resolverDescuento(comprador, codigoDescuento);
    // Recrear ítems con descuento aplicado
    List<ItemVenta> itemsConDescuento = new ArrayList<>();
    for (ItemVenta item : pedido.getItems()) {

```

```

        itemsConDescuento.add(new ItemVenta(
            item.getAlimento(), item.getCantidad(),
            item.getPrecioUnitario(), descuento
        ));
    }
    double subtotal = 0;
    for (ItemVenta item : itemsConDescuento) subtotal += item.getSubtotal();
    double impuesto = subtotal * 0.08;
    double propina = subtotal * pedido.getPropina();

    Venta venta = new Venta(idVenta, comprador, itemsConDescuento,
        new Date(), Venta.TipoVenta.CAFETERIA, subtotal, impuesto, propina);

    historialVentas.add(venta);

    if (comprador instanceof Cliente) {
        Cliente cliente = (Cliente) comprador;
        cliente.agregarPuntos(venta.calcularPuntosFidelidad());
        cliente.agregarCompra(venta);
    } else if (comprador instanceof Empleado) {
        ((Empleado) comprador).agregarPuntos(venta.calcularPuntosFidelidad());
    }
    return venta;
}

```

El método primero resuelve el descuento según quién está comprando: 20% si es un Empleado, 10% si presentó un código de descuento válido, 0% en cualquier otro caso. Estos descuentos no son acumulables. Luego recrea todos los ítems del pedido aplicando ese descuento a cada uno, de forma que el precio unitario queda guardado en el registro histórico (Para los registros que el Administrador necesita) con el descuento aplicado, no el precio original.

Con los ítems ya con descuento, calcula el subtotal. Sobre ese subtotal aplica el impuesto al consumo del 8% (no sobre el precio original). La propina se calcula también sobre el subtotal, usando el porcentaje que tenga configurado el pedido (default 10%, pero el cliente puede cambiarlo llamando `setPropina()` antes de finalizar, lo que sería preguntarle al cliente cuanto porcentaje desea de propina en el café).

Finalmente crea la venta, la agrega al historial, y llama a `agregarPuntos()` con el 1% del total, acreditándoselo al comprador, ya que el enunciado nos pide los puntos de fidelidad por cada compra, y estos si son acumulables

El detalle más importante: la propina se calcula sobre el subtotal antes de impuestos, no sobre el total final. Esto es importante porque si se calculara después, el cliente estaría "propinando" sobre el impuesto del gobierno, lo cual no tendría sentido. El orden es: subtotal, impuesto sobre subtotal, propina sobre subtotal, siendo así el total = subtotal + impuesto + propina.

12. Principios de diseño aplicados

El diseño fue construido aplicando conscientemente los principios y los conceptos de Responsibility-Driven Design estudiados durante el curso y con la información de las clases en las diapositivas.

12.1 Estereotipos y estilo de control (Responsibility-Driven Design)

Concepto	Aplicación en el diseño
Estilo de control: Delegado	SistemaCafe actúa como coordinador de alto nivel y delega las operaciones complejas a los servicios especializados. Los servicios a su vez delegan validaciones simples a los propios modelos (ej: juego.esAptoParaEdad()). Evita tanto el estilo centralizado como el disperso.
Information Holders	JuegoDeMesa, CopiaJuego, Mesa, Reserva, Turno, Prestamo, Venta, RegistroCompras — almacenan y entregan información.
Service Providers	ServicioPrestamo, ServicioVentas, ServicioCafeteria, ServicioTurnos, ServicioInventario, ServicioReportes, ServicioTorneo — ofrecen servicios complejos con lógica de negocio encadenada.
Coordinator	SistemaCafe — coordina entre servicios y mantiene el estado global del sistema.
Structurers	InventarioPrestamo, InventarioVenta — mantienen y organizan las relaciones entre objetos.

12.2 Principios SOLID:

El diseño fue construido respetando de forma consciente los cinco principios SOLID. A continuación, se documenta cómo se aplica cada uno en el sistema.

S — Single Responsibility	Cada clase tiene una razón única para cambiar. ServicioPrestamo solo gestiona préstamos; ServicioCafeteria solo gestiona el menú y pedidos; ServicioTurnos solo gestiona turnos. Si cambia la regla del máximo de préstamos activos por mesa, solo se modifica ServicioPrestamo (y Mesa). Si cambia el impuesto al consumo, solo se modifica ServicioCafeteria. La separación de InventarioPrestamo e InventarioVenta en dos clases distintas respeta este principio: uno cambia si cambia la lógica de copias físicas; el otro si cambia la lógica de stock y precios.
O — Open/Closed	Las jerarquías están abiertas para extensión y cerradas para modificación. Para agregar un nuevo tipo de alimento (p. ej. Snack), basta con crear una subclase de Alimento e implementar puedeServirseEnMesa(); ServicioCafeteria no necesita cambiar. Para el Proyecto 2, añadir torneoAmistoso y torneoCompetitivo como subclases de torneos siguió exactamente este principio: ServicioTorneo opera sobre torneos sin necesidad de modificarse para cada tipo. La jerarquía de Usuario (abstracta) → Cliente / Empleado / Administrador sigue el mismo patrón.
L — Liskov Substitution	Las subclases son sustituibles por sus clases base sin romper el sistema. ServicioPrestamo opera sobre Usuario sin importar si es

	Cliente, Mesero o Administrador (la restricción de Administrador se valida con <code>instanceof</code> antes de delegar, no dentro de la clase base). <code>ServicioCafeteria</code> opera sobre <code>Alimento</code> para llamar a <code>puedeServirseEnMesa()</code>; tanto <code>Bebida</code> como <code>Pasteleria</code> implementan este método correctamente. <code>ServicioTorneo</code> opera sobre torneos sin preocuparse de si es amistoso o competitivo para cupos e inscripciones; solo distingue el tipo en la lógica de pago y premios.
I — Interface Segregation	Aunque el proyecto no usa interfaces Java formales (no fue un requerimiento del curso), este principio se aplicó en el diseño de responsabilidades. <code>Cocinero</code> no tiene los métodos de <code>Mesero</code> (<code>conoceJuego</code>, <code>aprenderJuego</code>); <code>Administrador</code> no tiene los métodos de puntos de fidelidad de los empleados. Ninguna clase se ve forzada a implementar métodos que no le corresponden. La separación entre <code>ServicioReportes</code> (solo consulta) y <code>ServicioVentas</code> (solo registra ventas) evita que quien solo necesita reportes dependa de la lógica de ventas.
D — Dependency Inversion	Los módulos de alto nivel (<code>SistemaCafe</code>) no dependen de los detalles de bajo nivel; ambos dependen de abstracciones. <code>SistemaCafe</code> conoce a <code>ServicioPrestamo</code>, no a <code>InventarioPrestamo</code> directamente. <code>ServicioCafeteria</code> recibe <code>Alimento</code> (abstracto), no <code>Bebida</code> o <code>Pasteleria</code> concretas. La capa de persistencia (<code>PersistenciaCafe</code> y sus especializadas) es completamente invisible para los servicios: si el mecanismo de almacenamiento cambiara de archivos <code>.txt</code> a una base de datos, ningún servicio necesitaría modificarse. Esta inversión de dependencias se logra porque <code>SistemaCafe</code> es el único punto que conoce a <code>PersistenciaCafe</code>.

13. Restricciones del proyecto

13.1 Del café

- Capacidad máxima de clientes verificada en `registrarMesa()` sumando personas de todas las mesas ocupadas.
- Un cliente debe llamar `registrarMesa()` antes de poder pedir préstamos.
- Al registrar mesa, se indica número de personas, niños (< 5 años) y jóvenes (< 18 años).
- `liberarMesa()` devuelve todos los préstamos activos automáticamente (`Mesa.liberar()` llama `prestamo.devolver()`).

13.2 De préstamo de juegos

- Máximo 2 préstamos activos por mesa. Validado en `Mesa.agregarPrestamo()` y `ServicioPrestamo.validarPrestamo()`.
- La copia debe estar disponible en `InventarioPrestamo`. `getCopiaDisponible()` retorna la primera disponible del juego.
- El juego debe aceptar el número de personas en la mesa. `JuegoDeMesa.esAptoParaPersonas()`.
- Si hay niños: `edadMinima` mayor o igual a 5. Si hay jóvenes: `edadMinima` mayor o igual a 18. Validado en `esAptoParaEdad()`.
- Categoría ACCION + `tieneBebidaCaliente()` entonces bloqueado. Validado en ambas clases (`Mesa` y `ServicioPrestamo`).
- Juego difícil sin mesero capacitado: se presta con impresión de ADVERTENCIA. No se bloquea.
- Empleado en turno + hay clientes en el café entonces bloqueado. Empleado sin turno: siempre puede.
- Administrador: bloqueado en `SistemaCafe` antes de llegar al servicio.

13.3 De alimentos y bebidas

- Bebidas alcohólicas no se sirven a mesas con menores de 18 años.
- Bebidas calientes no se sirven a mesas con juego de Acción activo.
- Si el alimento tiene alérgenos, el sistema los informa antes de confirmar la compra.

13.4 De ventas

- IVA 19% para juegos (aplicado en `ServicioVentas.finalizarVenta()` como `subtotal * 0.19`). Impuesto al consumo 8% para cafetería (aplicado en `ServicioCafeteria.finalizarPedido()` como `subtotal * 0.08`).
- Descuento 20% automático para empleados. Código compartido da 10% a clientes. No acumulables (`resolverDescuento()`).
- Propina sugerida 10% del subtotal en cafetería. El cliente puede modificarla (aplicado en `setPropina()`).
- Todas las ventas generan 1% del total en puntos de fidelidad. 1 punto = 1 peso de descuento (`total * 0.01`). Se acumulan en `agregarPuntos()`.

13.5 De turnos

- Mínimo 1 cocinero y 2 meseros activos por día.
- Ningún empleado puede solicitar cambio que deje al café por debajo del mínimo.
- Intercambio solo entre empleados del mismo rol (mesero-mesero, cocinero-cocinero) y se hace por medio de `setTurno()` y `setEmpleado()`.
- Todos los cambios quedan en `solicitudesPendientes` hasta que el administrador los apruebe o rechace.

13.6 De torneos

- Cada torneo tiene un juego, día de la semana, cupo máximo y tipo: amistoso o competitivo.
- El cupo máximo depende de las copias disponibles del juego en `InventarioPrestamo`.

- Cada usuario puede inscribir máximo 3 participantes por torneo.
- Un usuario no puede inscribirse dos veces al mismo torneo.
- Al desinscribirse, se eliminan todos los cupos tomados por ese usuario.
- El 20% de los cupos se reserva para clientes fanáticos del juego, redondeado hacia arriba.
- Si se acaban los cupos fanáticos, los fanáticos toman cupos regulares.
- Los empleados solo pueden inscribirse si no están cubriendo turno ese día.
- En torneos competitivos, los clientes pagan tarifa y los empleados participan gratis, pero sin premio metálico.
- En torneos amistosos, el ganador recibe un bono de descuento no acumulable para una compra futura.

14. Historias de Usuario:

14.1 Administrador:

HU-A01 — Gestionar inventario de juegos

Como administrador, quiero consultar el estado de los inventarios (préstamo y venta), agregar copias, mover unidades entre inventarios y reabastecer stock, para mantener el café operativo.

Entrada: Juego a gestionar, cantidad, precio (si aplica), ID de copia (si aplica).

Salida: Confirmación de la operación o mensaje de error con la razón (sin stock, copia no existe, etc.).

Restricciones: Solo el administrador puede ejecutar estas operaciones. Si no hay stock en venta al intentar mover a préstamo, la operación es rechazada.

HU-A02 — Gestionar turnos del personal

Como administrador, quiero crear y modificar los turnos semanales del personal, y aprobar o rechazar solicitudes de cambio de turno, para garantizar que siempre haya al menos 1 cocinero y 2 meseros por día.

Entrada: Empleado, día de la semana. Para solicitudes: ID de la solicitud + decisión (aprobar/rechazar).

Salida: Turno creado/modificado, o solicitud aprobada/rechazada con el estado actualizado.

Restricciones: Si el cambio deja un día con menos del mínimo de personal, es rechazado automáticamente.

HU-A03 — Crear torneo (nuevo - Proyecto 2)

Como administrador, quiero crear torneos amistosos o competitivos para un juego específico, en un día de la semana, con un cupo máximo, para aumentar el flujo de clientes.

Entrada: Tipo de torneo (amistoso/competitivo), ID y nombre del torneo, juego, día de la semana, cupo máximo. Para amistoso: porcentaje del bono. Para competitivo: tarifa de entrada.

Salida: Torneo creado con confirmación, o mensaje de rechazo si las copias disponibles no soportan el cupo definido.

Restricciones: El cupo máximo no puede superar copias disponibles $\times$ maxJugadores del juego. Puede haber más de un torneo simultáneamente.

HU-A04 — Registrar ganador del torneo (nuevo - Proyecto 2)

Como administrador, quiero registrar al ganador de un torneo para que reciba su premio (bono de descuento o información del premio metálico).

Entrada: ID del torneo, usuario ganador.

Salida: Si amistoso: el bono queda guardado en el usuario. Si competitivo: se informa el monto del premio, o advertencia si el ganador es empleado y no puede recibirlo.

Restricciones: El ganador debe estar inscrito en el torneo.

14.2 Empleado:

HU-E01 — Solicitar cambio de turno

Como empleado, quiero solicitar un cambio o intercambio de turno, para poder gestionar mis compromisos personales sin afectar la operación del café.

Entrada: Tipo de solicitud (CAMBIO_GENERAL o INTERCAMBIO), turno afectado, empleado destino (si intercambio).

Salida: Solicitud creada en estado PENDIENTE, o rechazo si el cambio viola los mínimos de personal.

Restricciones: El intercambio solo puede hacerse con empleados del mismo rol. La solicitud queda pendiente hasta que el administrador la apruebe o rechace.

HU-E02 — Inscribirse a un torneo (nuevo - Proyecto 2)

Como empleado, quiero inscribirme a torneos de juegos cuando no tengo turno ese día, para participar en los eventos del café.

Entrada: ID del torneo, número de participantes (1–3).

Salida: Confirmación de inscripción con cupos asignados, o rechazo con la razón (turno ese día, sin cupos, ya inscrito).

Restricciones: Solo puede inscribirse si no tiene turno el día del torneo. En torneos competitivos participa gratis pero no puede recibir el premio en metálico.

HU-E03 — Desinscribirse de un torneo (nuevo - Proyecto 2)

Como empleado, quiero desinscribirme de un torneo en el que ya estoy inscrito, para liberar mis cupos si ya no puedo participar.

Entrada: Torneo al que está inscrito el empleado.

Salida: Confirmación de desinscripción y liberación de todos los cupos tomados por el empleado, o mensaje de error si no estaba inscrito.

Restricciones: Al desinscribirse se eliminan todos los cupos tomados por ese usuario en ese torneo.

14.3 Clientes:

HU-C01 — Reservar mesa

Como cliente, quiero reservar una mesa para mi grupo indicando el número de personas, niños y jóvenes, para poder disfrutar del café con mi grupo.

Entrada: Número de personas totales, cantidad de niños (<5 años), cantidad de jóvenes (<18 años).

Salida: Mesa asignada con confirmación, o mensaje de rechazo si el café está lleno o no hay mesas disponibles.

Restricciones: La suma de personas de todas las mesas ocupadas no puede superar la capacidad máxima del café.

HU-C02 — Inscribirse a un torneo (nuevo - Proyecto 2)

Como cliente, quiero inscribirme a torneos de juegos para competir y ganar premios o descuentos.

Entrada: ID del torneo, número de participantes (1–3).

Salida: Confirmación de inscripción indicando si se usaron cupos de fanático (si el juego está en mis favoritos) o cupos regulares. En torneos competitivos: monto total pagado.

Restricciones: Máximo 3 participantes por inscripción. Si el juego está en mis favoritos tengo prioridad en los cupos reservados (20% del cupo máximo, redondeado arriba). El descuento de torneo amistoso no es acumulable con otros descuentos.

HU-C03 — Desinscribirse de un torneo (nuevo - Proyecto 2)

Como cliente, quiero desinscribirme de un torneo en el que ya estoy inscrito, para liberar mis cupos si ya no voy a participar.

Entrada: Torneo al que está inscrito el cliente.

Salida: Confirmación de desinscripción y liberación de todos los cupos tomados por el cliente, o mensaje de error si no estaba inscrito.

Restricciones: Al desinscribirse se eliminan todos los cupos tomados por ese usuario en ese torneo.

15. Programas de prueba

Esta sección describe los programas de prueba diseñados para verificar el correcto funcionamiento del sistema. Los programas cubren los 27 requerimientos funcionales identificados, incluyendo los nuevos requerimientos de torneos. Las pruebas están organizadas en cinco programas temáticos que siguen el flujo real de uso del sistema.

15.1 Justificación de la consolidación en 6 programas

Los 23 requerimientos funcionales originales más los 4 nuevos de torneos (RF24–RF27) se consolidan en 6 programas de prueba. La siguiente tabla muestra las fusiones realizadas:

Pruebas originales fusionadas	Prueba consolidada	Justificación
Prueba 1 (inventario) + Prueba 2 (copias)	Prueba 1 – Gestión de inventario y copias	Ambas son operaciones del administrador sobre el mismo dominio. Gestionar stock y estado de

		copias forman un único flujo administrativo.
Prueba 3 (turnos) + aprobación admin	Prueba 2 – Gestión de turnos	Los turnos involucran exclusivamente a administrador y empleados. Combinarlos demuestra el flujo completo: asignar → consultar → solicitar → aprobar/rechazar.
Prueba 4 (menú) + Prueba 9 (informes)	Prueba 3 – Menú, sugerencias e informes	El administrador gestiona menú y consulta ventas en el mismo contexto operativo. Incluye RF05.
Prueba 5 (reservas+préstamos) + Prueba 6 (catálogo)	Prueba 4 – Flujo completo del cliente	El flujo del cliente es secuencial y continuo: llegar → reservar → consultar → pedir → ordenar → devolver.
Prueba 7 (compra juegos) + Prueba 8 (cafetería)	Prueba 5 – Ventas, descuentos y fidelidad	Ambas son transacciones económicas con los mismos mecanismos: descuentos, impuestos, puntos. Los puntos de una venta se pueden usar en la siguiente.
Prueba 9 (creación, inscripción y desinscripción a torneos) + Prueba 10 (ganador de torneo)	Prueba 6 – Gestión de torneos	Ambas pertenecen al mismo flujo del módulo de torneos: el administrador crea el torneo, los usuarios se inscriben o desinscriben y luego se registra el ganador con el premio correspondiente según sea amistoso o competitivo.

15.2 Descripción de los programas de prueba

Prueba 1 — Gestión de inventario y copias

Actor(es): Administrador - RFs cubiertos: RF01, RF03 - CUMPLE

El código de ServicioInventario implementa correctamente todos los escenarios de esta prueba:

- Consulta de estado de inventarios de préstamo y venta (getCopiasPorJuego, getInventarioPrestamo, getInventarioVenta).
- Agregar copias al inventario de préstamo: agregarCopiaAPrestamo(copia).
- Mover unidad del inventario de venta al de préstamo: moverDeVentaAPrestamo() — verifica hayStock antes de proceder; retorna false si no hay stock.

- Cambiar estado físico de una copia: `actualizarEstadoCopia(copia, nuevoEstado)`.
- Marcar una copia como desaparecida: `marcarComoRobada()` actualiza estado a 'Desaparecido' y la marca no disponible.
- Reparar una copia: `repararCopia()` descuenta una unidad del inventario de venta, actualiza estado a 'Nuevo' y la marca disponible.

Escenarios cubiertos:

- Movimiento exitoso de venta a préstamo cuando hay stock.
- Rechazo del movimiento cuando no hay stock en inventario de venta (retorna false).
- Reparación exitosa consumiendo una unidad de venta.
- Marcado de copia como robada/desaparecida con liberación de disponibilidad.

Prueba 2 — Gestión de turnos

Actor(es): Administrador y Empleados - RFs cubiertos: RF02, RF15, RF16 - CUMPLE

El código de `ServicioTurnos` implementa correctamente todos los escenarios:

- Crear turno: `crearTurno()` asigna el turno al empleado (`setTurno()`) y lo registra en la lista.
- Consulta de turnos propios: `getTurnos()` devuelve la lista completa; se filtra por empleado en la consola.
- Solicitar cambio: `solicitarCambio()` valida mínimos antes de agregar a `solicitudesPendientes`; rechaza si viola mínimos.
- Aprobar solicitud: `aprobarSolicitud()` re-valida mínimos. Si `INTERCAMBIO`, hace el swap de `setTurno()/setEmpleado()`. Llama `solicitud.aprobar()`.
- Rechazar solicitud: `rechazarSolicitud()` cambia estado a `RECHAZADA` y la remueve de pendientes.
- Intento de cambio que viola mínimos (< 1 cocinero o < 2 meseros): bloqueado tanto en `solicitarCambio` como en `aprobarSolicitud`.

Escenarios cubiertos:

- Asignación y consulta de turnos por empleado.
- Solicitud de `CAMBIO_GENERAL` bloqueada por mínimos.
- Solicitud de `INTERCAMBIO` exitosa entre meseros.
- Aprobación y rechazo de solicitudes pendientes.

Prueba 3 — Menú, sugerencias e informes

Actor(es): Administrador y Empleados - RFs cubiertos: RF04, RF05, RF17 - CUMPLE

El código de ServicioCafeteria y ServicioReportes implementa correctamente los escenarios:

- Agregar alimento directo al menú: agregarAlMenu(alimento).
- Sugerencias: Sugerencia.aprobar() cambia estado a APROBADA; rechazar() a RECHAZADA.
- Informes filtrados: generarInformeVentas(tipo, desde, hasta) filtra por TipoVenta y rango de fechas.
- Informes por período: ventasPorDia(), ventasPorSemana(), ventasPorMes().
- Cálculo de totales: getTotalSubtotal(), getTotalImpuestos(), getTotalPropinas().

Escenarios cubiertos:

- Registro de bebida (alcoholica=true, caliente=false) y pastelería (con alérgenos) al menú.
- Flujo completo de sugerencia: empleado propone → admin aprueba (queda en menú) o rechaza.
- Generación de informe CAFETERIA del día actual con desglose de subtotal, impuestos y propinas.
- Informe mensual con múltiples ventas registradas.

Prueba 4 — Flujo completo del cliente

Actor(es): Cliente y Mesero - RFs cubiertos: RF06, RF07, RF08, RF09, RF11, RF13, RF14, RF18, RF22 - CUMPLE

El código de SistemaCafe, ServicioPrestamo y Mesa implementa correctamente todos los escenarios:

- Registro de mesa con validación de capacidad máxima (SistemaCafeTest lo verifica con dos escenarios: exitoso y rechazado por capacidad/sin mesas libres).
- Catálogo de juegos con disponibilidad en tiempo real: InventarioPrestamo.getCopiaDisponible() retorna null si no hay copias disponibles.
- Préstamo con todas las restricciones encadenadas en ServicioPrestamo.validarPrestamo(): administrador bloqueado, empleado con clientes bloqueado, máx. 2 préstamos activos, jugadores, edad, bebida caliente + Accion.
- Juego difícil sin mesero capacitado: préstamo permitido con ADVERTENCIA impresa, no bloqueado.
- Alimentos validados por puedeServirseEnMesa(): Bebida bloquea alcohol con menores, bloquea caliente con Accion; Pasteleria informa alérgenos sin bloquear.
- Favoritos: agregarFavorito() y quitarFavorito() en Usuario (heredado por Cliente y Empleado).
- Liberación de mesa: Mesa.liberar() devuelve todos los préstamos activos automáticamente.

Escenarios cubiertos:

- Reservar mesa exitosa, rechazada por capacidad y rechazada por sin mesas libres (cubierto en SistemaCafeTest).
- Intento de tercer préstamo bloqueado (límite 2 activos).
- Juego de Acción bloqueado con bebida caliente en la mesa.
- Juego con restricción de edad bloqueado con niños presentes.
- Préstamo de juego difícil con y sin mesero disponible.
- Pedido de bebida alcohólica rechazado en mesa con menores.
- Liberación de mesa y devolución automática de préstamos activos.

Prueba 5 — Ventas, descuentos y fidelidad

Actor(es): Cliente y Empleado - RFs cubiertos: RF10, RF12, RF19, RF20, RF21, RF23 - CUMPLE

Esta prueba es la más cubierta por los tests JUnit implementados. El código de ServicioVentas verifica:

- Cálculo correcto de IVA 19% sobre el subtotal (verificado en finalizarVenta_calculaSubtotalIVAyPuntos).
- Descuento 20% automático para empleados (verificado en resolverDescuento_empleado_codigo_sinDescuento).
- Descuento 10% con código compartido para clientes (verificado en el mismo test).
- Sin descuento cuando no hay código ni es empleado (0.0, verificado en el mismo test).
- Descuento de stock al finalizar la venta (verificado en finalizarVenta_reducesStockInventario).
- Acumulación de puntos de fidelidad: 1% del total (verificado en finalizarVenta_calculaSubtotalIVAyPuntos).
- Uso de puntos insuficientes rechazado sin descontar nada (verificado en ClienteTest.usarPuntos_exitoso_YFallaConInsuficientes).
- Autenticación exitosa y fallida (verificado en SistemaCafeTest.login_exitoso_YFallidoConCredencialesInvalidas).
- Restricción de venta al administrador (verificado en SistemaCafeTest.realizarVenta_rechazada_administradorNoPuedeComprarNiOrdenar).

Escenarios cubiertos:

- Venta de juego con IVA y generación de puntos (RF10, RF23).
- Venta con código de empleado al 10% (RF10).
- Venta de empleado al 20% automático (RF10).
- Uso de puntos acumulados como descuento en compra siguiente (RF12).
- Ventas de cafetería con impuesto al consumo 8% y propina (RF11).
- Autenticación completa (RF21).

Prueba 6 — Gestión de torneos

Actor(es): Administrador, Clientes y Empleados - RFs cubiertos: RF24, RF25, RF26, RF27 - CUMPLE

Las pruebas JUnit implementadas para torneos verifican el flujo completo del módulo: creación de torneos, inscripción/desinscripción, cupos fanáticos, restricciones de empleados y registro de ganadores.

- Crear torneos amistosos y competitivos con juego, día, cupo máximo y premio/tarifa correspondiente.
- Validar que el cupo máximo no supere la capacidad permitida por las copias disponibles del juego.
- Inscribir usuarios con máximo 3 participantes por torneo.
- Evitar inscripciones duplicadas al mismo torneo.
- Separar cupos fanáticos y regulares.
- Reservar el 20% de los cupos para clientes fanáticos del juego.
- Permitir que fanáticos tomen cupos regulares si se acaban los cupos reservados.
- Bloquear empleados que estén cubriendo turno el día del torneo.
- Permitir empleados fuera de turno, con inscripción gratuita en torneos competitivos.
- Desinscribir usuarios liberando todos sus cupos.
- Registrar ganadores y asignar el premio correspondiente: bono en amistosos o premio metálico en competitivos.
- Impedir que empleados reciban premio metálico en torneos competitivos.

Escenarios cubiertos:

- Creación exitosa de torneo cuando hay suficientes copias disponibles.
- Rechazo de torneo cuando el cupo supera la capacidad permitida por las copias.
- Inscripción de cliente fanático usando cupo reservado.
- Inscripción de cliente no fanático usando cupo regular.
- Rechazo de inscripción duplicada.
- Rechazo de inscripción con más de 3 participantes.
- Desinscripción exitosa liberando cupos.
- Rechazo de empleado con turno el día del torneo.
- Inscripción de empleado fuera de turno.
- Competitivo: cliente paga, empleado participa gratis y no recibe premio metálico.
- Amistoso: ganador recibe bono de descuento.
- Rechazo de registrar ganador si el usuario no está inscrito.

15.3 Justificación de los tests implementados (JUnit 5)

Los siguientes tests fueron implementados por el equipo y verifican los métodos con mayor lógica de negocio. Se excluyeron conscientemente getters, setters y constructores simples, siguiendo las indicaciones del proyecto.

ClienteTest — usuarios.ClienteTest

Test	RF cubierto	Qué verifica
usarPuntos_exitoso_YFallaConInsuficientes()	RF12 — Programa de fidelidad	Que usarPuntos() descuenta correctamente el saldo ($500 - 300 = 200$) cuando hay suficientes puntos, devuelve true; y que con puntos insuficientes (intentar usar 500 teniendo 200) devuelve false sin modificar el saldo. Es la lógica núcleo del programa de fidelidad.

ServicioVentasTest — Servicios.ServicioVentasTest

Test	RF cubierto	Qué verifica
finalizarVenta_calculaSubtotalIVAyPuntos()	RF10 — Compra juego RF23 — Fidelidad	Que al finalizar una venta de tienda: subtotal = 50.000, IVA 19% = 9.500, total = 59.500, puntos acumulados = 595 (1% del total). Verifica las tres fórmulas financieras centrales de la tienda en una sola transacción.
resolverDescuento_empleado_codigo_sinDescuento()	RF10 — Descuentos	Que resolverDescuento() retorna exactamente: 0.20 para Mesero (empleado), 0.10 para Cliente con código válido, 0.0 para Cliente sin código. Valida las tres rutas del método de forma aislada sin

		necesidad de una venta completa.
finalizarVenta_reducesStockInventario()	RF10 — Stock	Que al comprar 2 unidades de un juego con stock inicial de 5, el stock queda en 3. Verifica que la operación financiera (finalizar venta) y la operación de inventario (descontar stock) están acopladas correctamente en finalizarVenta().

SistemaCafeTest — sistema.SistemaCafeTest

Test	RF cubierto	Qué verifica
registrarMesa_exitosa_YRechazadaPorCapacidad()	RF13 — Reservar mesa	Que registrarMesa() asigna una mesa válida (no null, isOcupada=true) con 4 personas sobre una capacidad de 10; y que un segundo cliente con 8 personas es rechazado (null) porque $4+8=12 > 10$. Verifica el control de capacidad máxima del café.
registrarMesa_rechazada_noHayMesasLibres()	RF13 — Reservar mesa	Que si las 2 mesas físicas del sistema están ocupadas, un tercer cliente recibe null aunque quede capacidad de personas disponible. Verifica que el sistema distingue entre capacidad de personas y disponibilidad física de mesas.

login_exitoso_YFallidoConCredencialesInvalidas()	RF21 — Autenticación	Que login() retorna el usuario correcto con login+password válidos (not null); retorna null con password incorrecto ('semeolvinox'); y retorna null con login inexistente ('Tontolio'). Cubre los tres casos posibles de autenticación.
realizarVenta_rechazada_administradorNoPuedeComprarNiOrdenar()	RF10, RF11 — Restricción Admin	Que el administrador recibe null al intentar realizarVentaTienda() y al intentar realizarVentaCafeteria(). Verifica el control de permisos de SistemaCafe antes de llegar a cualquier servicio, cubriendo la restricción más importante del rol Admin.
flujoTorneoAmistoso_desdeSistemaCafe_guardaBonoGanador	RF24, RF25, RF26	Verifica el flujo completo desde SistemaCafe: crear torneo amistoso, inscribir cliente y registrar ganador con bono.
flujoTorneoCompetitivo_desdeSistemaCafe_clientePagaEmpleadoGratis	RF24, RF25, RF26	Verifica el flujo completo desde SistemaCafe: crear torneo competitivo, inscribir cliente y empleado, calcular premio y validar que el empleado no reciba premio metalico.

Test	RF cubierto	Qué verifica
crearTorneo_exitoso_yFallaSiNoHayCopiasSuficientes()	RF24 — Crear torneo	Verifica que un torneo se cree cuando las copias disponibles soportan el cupo, y que se rechace si el cupo supera la capacidad posible.
inscribirse_fanaticoUsaCupoReservado_yNormalUsaRegular()	RF26 — Inscribirse a torneo	Verifica que un cliente fanático use cupos reservados y que un cliente no fanático use cupos regulares.
inscribirse_fallaDuplicado_yMasDeTresParticipantes()	RF26 — Inscribirse a torneo	Verifica que un usuario no pueda inscribirse dos veces al mismo torneo ni inscribir más de 3 participantes.
desinscribirse_eliminaTodosLosCuposDelUsuario()	RF27 — Desinscribirse de torneo	Verifica que al desinscribirse se se eliminen todos los cupos tomados

Test	RF cubierto	Qué verifica
		por ese usuario.
empleadoEnTurnoMismoDiaNoPuedeInscribirse_peroSinTurnoActivoSiPuede()	RF26 — Inscribirse a torneo	Verifica que un empleado con turno el día del torneo sea rechazado y que un empleado sin turno activo pueda inscribirse.
empleadoNoUsaCupoFanaticoAunqueTengaJuegoFavorito()	RF26 — Cupos fanáticos	Verifica que los cupos fanáticos sean exclusivos para clientes, no para empleados.
competitivo_clientePaga_empleadoGratisYNoRecibePremio()	RF25/RF26 — Torneo competitivo	Verifica que el cliente pague tarifa, el empleado participe gratis y no pueda recibir premio metálico.
registrarGanadorAmistoso_guardaBonoEnCliente()	RF25 — Registrar ganador	Verifica que el ganador de un torneo amistoso reciba el

Test	RF cubierto	Qué verifica
		bono de descuento.
registrarGanador_fallaSiUsuarioNoEstaInscrito()	RF25 — Registrar ganador	Verifica que no se pueda registrar como ganador a un usuario no inscrito.

15.4 Resultados de los tests implementados

A continuación se presentan los resultados de los tests JUnit implementados por el equipo en esta entrega. Todos los tests existentes pasan exitosamente.

Clase	Test	Resultado	Observación
ClienteTest	usarPuntos_exitoso_YFallaConInsuficientes	PASA	Lógica de puntos completamente correcta. Saldo final verificado en ambos casos.
ServicioVentas Test	finalizarVenta_calculaSubtotalIVAyPuntos	PASA	Subtotal, IVA 19% y puntos 1% calculados con delta 0.01. Formulas correctas.
ServicioVentas Test	resolverDescuento_empleado_codigo_sinDescuento	PASA	Las tres rutas de descuento retornan 0.20, 0.10 y 0.0 respectivamente.
ServicioVentas Test	finalizarVenta_reducesStockInventario	PASA	Stock de 5 unidades queda en 3 después de comprar 2. Acoplamiento correcto.
SistemaCafeTest	registrarMesa_exitosa_YRechazadaPorCapacidad	PASA	Mesa asignada con 4 personas; rechazada con 8 adicionales sobre capacidad 10.
SistemaCafeTest	registrarMesa_rechazada_noHayMesasLibres	PASA	Tercer cliente rechazado cuando

			las 2 mesas físicas están ocupadas.
SistemaCafeTest	login_exitoso_YFallidoConCredencialesInvalidas	PASA	Retorna usuario válido, null con mala contraseña, null con login inexistente.
SistemaCafeTest	realizarVenta_rechazada_administradorNoPuedeComprarNiOrdenar	PASA	Admin bloqueado en tienda y cafetería. Control de permisos correcto.

Clase	Test	Resultado	Observación
ServicioTorneoTest	crearTorneo_exitoso_yFallaSiNoHayCopiasSuficientes	PASA	Crea torneo válido y rechaza torneo con cupo mayor al permitido por las copias disponibles.
ServicioTorneoTest	inscribirse_fanaticoUsaCupoReservado_yNormalUsaRegular	PASA	Valida que el fanático use cupo reservado y el no fanático use cupo regular.
ServicioTorneoTest	inscribirse_fallaDuplicado_yMasDeTresParticipantes	PASA	Bloquea inscripción duplicada y más de 3 participantes por usuario.
ServicioTorneoTest	desinscribirse_eliminaTodosLosCuposDelUsuario	PASA	Libera todos los cupos tomados por el usuario al desinscribirse.
ServicioTorneoTest	empleadoEnTurnoMismoDiaNoPuedeInscribirse_peroSinTurnoActivoSiPuede	PASA	Rechaza empleado con turno el

			día del torneo y permite empleado sin turno activo.
ServicioTorneoTest	empleadoNoUsaCupoFanaticoAunqueTengaJuegoFavorito	PASA	Confirma que los cupos fanáticos son solo para clientes.
ServicioTorneoTest	competitivo_clientePaga_empleadoGratisYNoRecibePremio	PASA	Cliente paga tarifa, empleado participa gratis y no recibe premio metálico.
ServicioTorneoTest	registrarGanadorAmistoso_guardaBonoEnCliente	PASA	El ganador de torneo amistoso recibe y guarda el bono de descuento.
ServicioTorneoTest	registrarGanador_fallaSiUsuarioNoEstaInscrito	PASA	Rechaza registrar como ganador a un usuario no inscrito.
SistemaCafeTest	flujoTorneoAmistoso_desdeSistemaCafe_guardaBonoGanador	PASA	Prueba de integración: crea torneo amistoso, inscribe cliente y guarda bono al ganador.
SistemaCafeTest	flujoTorneoCompetitivo_desdeSistemaCafe_clientePagaEmpleadoGratis	PASA	Prueba de integración: crea torneo competitivo, inscribe cliente/empleado y

			valida premio.
--	--	--	----------------

16. Consola Implementada:

El paquete consola implementa la interfaz de usuario por línea de comandos del sistema. Permite que tres tipos de usuario, Cliente, Empleado y Administrador, interactúen con el sistema cada uno desde su propio punto de entrada (main), con autenticación obligatoria al iniciar.

Arquitectura

El paquete se compone de cuatro clases:

Clase	Rol
ConsolaBase	Clase base abstracta. Centraliza la carga/guardado de datos, la autenticación y los métodos de lectura con validación (leerEntero, leerTexto, etc.). Las tres consolas la extienden.
ConsolaCliente	Punto de entrada para clientes. Incluye registro propio antes del login.
ConsolaEmpleado	Punto de entrada para empleados (meseros y cocineros).
ConsolaAdministrador	Punto de entrada para el administrador.

Cada consola tiene su propio main(). Al iniciarse, carga automáticamente los datos desde archivos; al salir, los guarda. El usuario no escoge ni escribe rutas de archivo en ningún momento.

ConsolaCliente

Antes del login se muestra un menú previo con dos opciones: **iniciar sesión** o **registrarse**. Un cliente nuevo puede crear su cuenta directamente, sin necesidad de que el administrador lo haga por él.

#	Opción	Descripción
1	Registrar mesa	Solicita número de personas (adultos, niños, jóvenes). El sistema asigna una mesa libre si hay capacidad.
2	Liberar mesa	Desocupa la mesa asignada al cliente.
3	Ver catálogo de juegos	Muestra todos los juegos con copias disponibles para préstamo y precio de venta.

4	Pedir préstamo de juego	Lista los juegos disponibles. Requiere tener mesa asignada.
5	Devolver juego prestado	Muestra los juegos actualmente en préstamo en la mesa del cliente.
6	Realizar pedido en cafetería	Permite seleccionar ítems del menú. Pregunta el porcentaje de propina (10% sugerido, editable). Aplica código de descuento o bono de torneo si corresponde.
7	Comprar juego en tienda	Permite comprar juegos del inventario de venta. Acepta código de descuento.
8	Ver mis puntos de fidelidad	Muestra el saldo de puntos y el bono de torneo amistoso pendiente (si hay).
9	Ver / agregar juegos favoritos	Submenú para consultar, agregar o quitar juegos de la lista personal de favoritos.
10	Ver torneos disponibles	Lista todos los torneos con tipo, juego, día y cupos.
11	Inscribirse a torneo	Permite inscribirse en un torneo con cupo disponible, indicando número de participantes.
12	Desinscribirse de torneo	Cancela la inscripción en un torneo en el que el cliente esté inscrito.
13	Salir	Cierra sesión y guarda los datos.

En compras y pedidos se pregunta si el cliente desea usar puntos de fidelidad; el descuento se muestra como neto separado sin modificar el total oficial de la venta.

ConsolaEmpleado

#	Opción	Descripción
1	Ver mi turno actual	Muestra el ID y día del turno asignado, o indica que no tiene turno.
2	Solicitar cambio de turno	Permite solicitar liberación del turno o intercambio con otro empleado que tenga turno. La solicitud queda pendiente de aprobación.
3	Pedir préstamo de juego	Lista juegos disponibles. Bloqueado si el empleado está en turno y hay clientes en el café. No requiere mesa propia.
4	Devolver juego prestado	Muestra los préstamos activos del empleado y permite devolver uno.

5	Realizar pedido en cafetería	El empleado selecciona una mesa ocupada para la que toma el pedido. El descuento del 20% se aplica automáticamente.
6	Comprar juego en tienda	Igual que el cliente, con descuento de empleado aplicado automáticamente.
7	Sugerir ítem al menú	Envía una sugerencia de nuevo producto con nombre y categoría. Queda pendiente de aprobación del administrador.
8	Ver mis puntos de fidelidad	Muestra saldo de puntos y bono de torneo pendiente.
9	Ver / agregar juegos favoritos	Igual que en cliente.
10	Inscribirse a torneo	Igual que en cliente. En torneos competitivos se indica que la entrada es gratuita y sin premio en efectivo.
11	Desinscribirse de torneo	Igual que en cliente.
12	Salir	Cierra sesión y guarda los datos.

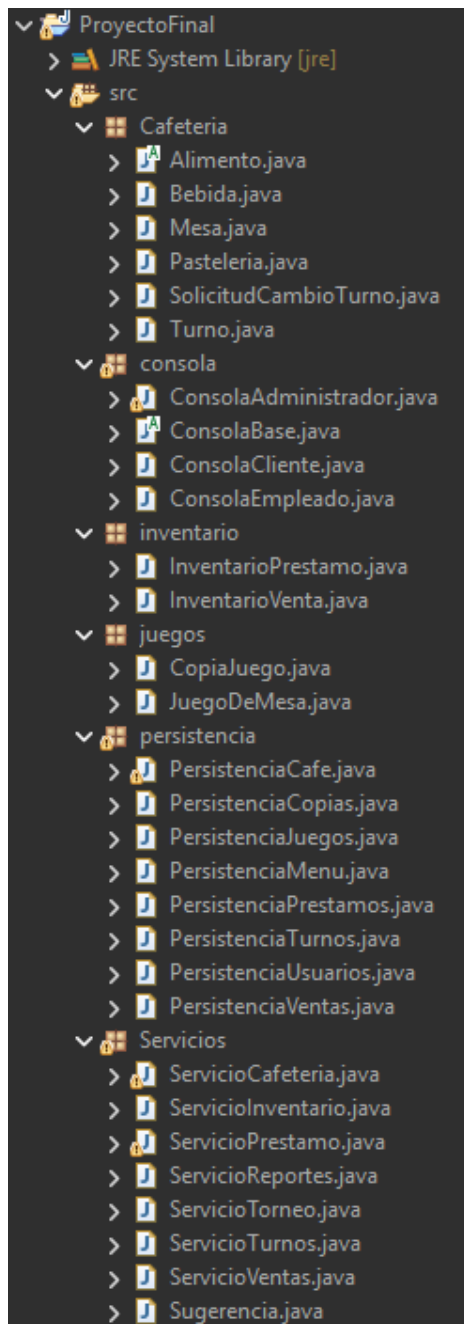
ConsolaAdministrador

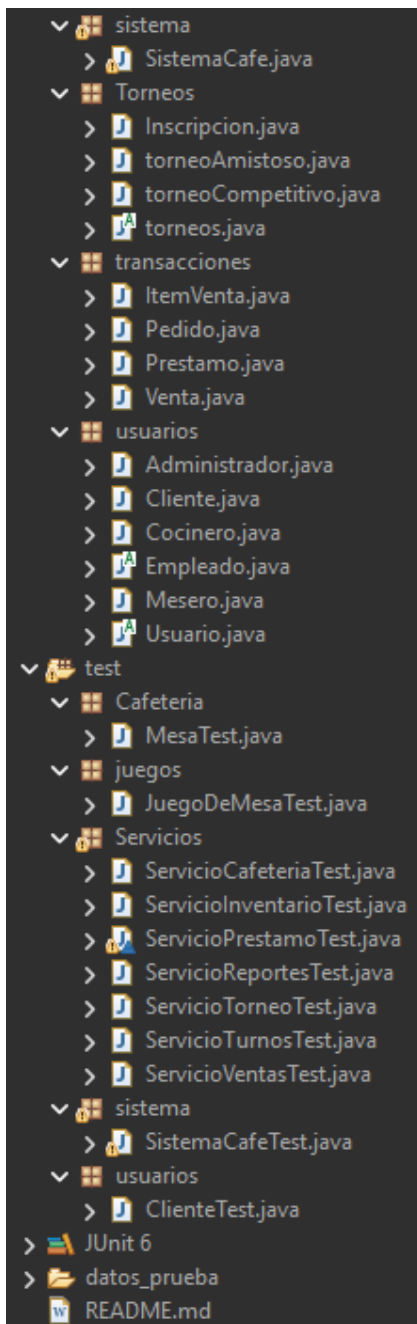
#	Opción	Descripción
1	Agregar usuario	Crea un nuevo Cliente, Mesero o Cocinero con todos sus datos.
2	Listar usuarios	Muestra tabla con ID, nombre, tipo y login de todos los usuarios.
3	Agregar juego al inventario	Registra un juego nuevo con todos sus atributos y lo agrega al inventario de préstamo, venta o ambos.
4	Gestionar inventario de juegos	Submenú: ver copias, mover unidad de venta a préstamo, reparar copia, marcar copia como robada, reabastecer venta.
5	Agregar ítem al menú	Crea una Bebida (con indicadores de alcohólica/caliente) o Pastelería (con lista de alérgenos) y la agrega al menú.
6	Construir / modificar turno	Submenú: ver todos los turnos, crear turno asignando empleado y día, eliminar turno existente.
7	Ver solicitudes de cambio de turno	Lista las solicitudes pendientes con solicitante, tipo y empleado destino.
8	Aprobar / Rechazar solicitud de turno	Selecciona una solicitud pendiente y la aprueba o rechaza.

9	Ver sugerencias de menú	Lista todas las sugerencias con estado (pendiente / aprobada / rechazada).
10	Aprobar / Rechazar sugerencia	Resuelve una sugerencia pendiente. Si se aprueba, ofrece la opción de agregar el ítem al menú de inmediato.
11	Crear torneo	Crea un torneo Amistoso (con porcentaje de bono para el ganador) o Competitivo (con tarifa de entrada).
12	Ver torneos	Lista todos los torneos con tipo, juego, día, cupos y condiciones económicas.
13	Registrar ganador de torneo	Selecciona un torneo e identifica al ganador entre los inscritos. El sistema le aplica automáticamente el bono o el premio según el tipo de torneo.
14	Ver reporte de ventas	Submenú: reporte por tipo y rango de fechas, por semana o por mes. Muestra subtotal, impuestos, propinas y total por separado, tanto por fila como en el resumen final.
15	Salir	Cierra sesión y guarda los datos.

17. Estructura del proyecto Eclipse

Además de la ayuda de los diagramas, a continuación, ponemos la estructura del proyecto desde la vista de ECLIPSE.





18. Preocupaciones transversales

Esta sección documenta aspectos que cruzan múltiples componentes y que requieren atención especial tanto en el diseño como en el uso de la implementación del sistema

18.1 Orden de llamadas crítico en SistemaCafe

El mayor riesgo operativo del sistema está en el orden en que se inicializa y se usa SistemaCafe. Si no se respeta, el comportamiento puede ser inesperado:

Con esto, nos referimos a que el constructor de SistemaCafe extrae meseros y empleados de la lista de usuarios ANTES de que se hayan agregado. Esto significa que ServicioPrestamo y ServicioTurnos arrancan con listas vacías. Siempre usar agregarUsuario() para agregar empleados y meseros DESPUÉS de construir el SistemaCafe.

Paso	Qué hace / por qué importa el orden
1. Crear SistemaCafe	new SistemaCafe(capacidadMaxima) - servicios se inicializan aquí
2. Agregar mesas	agregarMesa() por cada mesa física del café
3. Agregar usuarios	agregarUsuario() por cada cliente, mesero, cocinero y admin. ESTO agrega a los servicios correctos.
4. Cargar inventario	servicioInventario.reabastecerVenta() / agregarCopiaAPrestamo()
5. Cargar menú	agregarAlimento() para bebidas y pastelerías
6. Asignar turnos	servicioTurnos.crearTurno() para construir la semana
7. Operación normal	login(), registrarMesa(), prestarJuego(), etc.

18.2 Doble validación en el flujo de préstamo

Actualmente las validaciones de préstamo están duplicadas entre ServicioPrestamo.validarPrestamo() y Mesa.agregarPrestamo(). El flujo real es:

1. validarPrestamo() ← valida TODO (jugadores, edad, Accion+caliente, etc.)
2. getCopiaDisponible() ← busca copia física
3. mesa.agregarPrestamo() ← valida TODO otra vez

Consecuencia: si las dos validaciones difieren, una puede aprobar y la otra rechaza, simultáneamente en mesa o en ServicioPrestamo.

Sin embargo, al mantener sincornizadas las dos validaciones, el sistema funciona y seguimos manteniendo la doble validación por si acaso.

18.3 Riesgo en la gestión de inventarios por el administrador

El administrador tiene acceso directo a operaciones que pueden dejar el sistema en estado inconsistente si se usan en el orden incorrecto:

Operación	Riesgo y cómo mitigarlo
marcarComoRobada() sobre copia activa	Si se marca como robada una copia que está actualmente prestada (activo=true), el préstamo sigue existiendo, pero la copia ya no estará disponible al devolver. Siempre verificar <code>prestamo.isActivo()</code> antes.
repararCopia() sin verificar stock	<code>repararCopia()</code> devuelve false si no hay stock en venta, pero si se ignora el retorno, se podría continuar asumiendo que la reparación se hizo. Siempre verificar el valor de retorno.
moverDeVentaAPrestamo() con juego en préstamo activo	Mover todas las copias de un juego podría dejar sin stock de venta al café inesperadamente. No hay validación que lo prevenga, por lo que el administrador siempre debe estar consciente del inventario que está manejando.

18.4 Integridad del historial

Los préstamos y ventas nunca se eliminan del sistema. Los préstamos solo cambian el flag activo al devolverse; las ventas son inmutables. `ItemVenta` guarda el `precioUnitario` en el momento de la compra, por lo que el historial es correcto aunque el precio del juego cambie después.

19. Uso de Inteligencia Artificial Generativa:

A continuación, se documenta un poco el uso realizado de herramientas de inteligencia artificial generativa durante el desarrollo del proyecto 2:

Campo	Detalle
Herramienta	Claude (Anthropic)
Versión del modelo	Claude Sonnet 4.6 (claude-sonnet-4-6)
Nivel de uso autorizado	Nivel 3 — Uso para editar y refinar (no para generar funcionalidades críticas)
Propósito principal	Refinamiento de redacción del documento de diseño y orientación sobre priorización de tests

La herramienta fue utilizada para mejorar el lenguaje del documento de diseño ya redactado por el equipo para mejorar claridad y consistencia (junto con tonos formales y definiciones más acordes al curso). En ningún caso se generó contenido técnico desde cero; todos los contenidos (diagramas, decisiones, APIs, requerimientos) fueron elaborados primero por el equipo y luego refinados en redacción y estilo.

Ejemplos de prompts utilizados para refinamiento:

Prompt utilizado (extracto)	Propósito del refinamiento
"La descripción de la Prueba 4 en la sección 14 cumple con los requerimientos especificados por nosotros? La idea es no cambiar la prueba, ya que es la original, pero queremos estar conscientes de nuestros límites"	Verificar coherencia entre la descripción del programa de prueba y el código implementado.
"En el análisis de SOLID para la sección 12, la justificación del principio open menciona suficientes ejemplos del sistema o se queda muy abstracta? Que ejemplo concreto del código puedo añadir para hacerlo mas convincente?"	Enriquecer la argumentación del análisis SOLID con ejemplos concretos del sistema.

La herramienta fue utilizada para orientar al equipo sobre cuáles métodos y flujos debían priorizarse para implementar tests, dado que no era viable ni necesario testear todos los métodos del sistema (getters, setters y constructores fueron excluidos explícitamente por las instrucciones del proyecto).

Prompts utilizados:

Prompt utilizado (extracto)	Decisión tomada a partir de la orientación
"Nuestro proyecto pide pruebas unitarias para la lógica de la aplicación y al menos una prueba por requerimiento funcional, cuales son los metodos de nuestro sistema que tienen mas lógica y que valdría la pena cubrir con un test unitario?"	Se priorizaron: <code>resolverDescuento()</code> , <code>finalizarVenta()</code> , <code>usarPuntos()</code> y <code>registrarMesa()</code> por concentrar la mayor cantidad de lógica de negocio con múltiples rutas de ejecución.

Prompt utilizado (extracto)	Decisión tomada a partir de la orientación
“El enunciado pide pruebas de integración por historias de usuario. Tiene sentido incluir pruebas de torneos en SistemaCafeTest?”	Se agregaron pruebas de integración en SistemaCafeTest para validar el flujo completo de torneo amistoso y competitivo desde el sistema principal.

Prompt utilizado (extracto)	Decisión tomada a partir de la orientación
"quiero que el menú de las consolas se vea ordenado y profesional, con separadores visuales y que las opciones estén numeradas claramente"	Se estandarizó la presentación de todos los menús del sistema utilizando bordes con <u>=====</u> , títulos centrados y opciones numeradas con formato uniforme, aplicado desde ConsolaBase para que las tres consolas compartan la misma estética visual.

Prompt utilizado (extracto)	Decisión tomada a partir de la orientación
“Ayúdanos a redactar mejor el README con instrucciones de uso de las consolas y ejecución de pruebas JUnit.”	Se utilizó la IA como apoyo de documentación para organizar el README en formato de manual de usuario, sin modificar la lógica del sistema.